

UNDERGRADUATE FINAL YEAR PROJECT REPORT

Department of Computer and Information Systems Engineering

NED University of Engineering and Technology



See Beyond

Group Number: 11

Batch: 2022

Group Member Names:

Sheikh Malaika Mustafa

CS-22006

Sumaika Imran

CS-22009

QurratulAin

CS-22018

Urooj Fatima

CS-22022

Approved by

.....

Dr. Urooj Ainuddin

Assistant Professor

Project Advisor

Author's Declaration

We declare that we are the sole authors of this project. It is the actual copy of the project that was accepted by our advisor(s) including any necessary revisions. We also grant NED University of Engineering and Technology permission to reproduce and distribute electronic or paper copies of this project.

15 June 2026

Sheikh Malaika Mutaifa

CS-22006

mustafa4504116@cloud.neduet.edu.pk

15 June 2026

QurratulAin

CS-22018

ain4503262@cloud.neduet.edu.pk

15 June 2026

Sumaika Imran

CS-22009

imran4504279@cloud.neduet.edu.pk

15 June 2026

Urooj Fatima

CS-22022

fatima4503560@cloud.neduet.edu.pk

Statement of Contributions

- All authors contributed equally to the research, planning, and writing of this Final Year Project report, participating in literature review, system analysis and revision.
- All group members collaboratively evaluated existing assistive technologies and computer vision models relevant to visually impaired assistance systems.
- **Sheikh Malaika Mustafa** was responsible for the **backend system** and the **integration** of multiple AI models. She set up the **face recognition** pipeline, which included detecting faces, aligning them, pulling features, and matching identities. To make sure the mobile app could talk to the system smoothly, she built **RESTful APIs** using **Fast API**. She also put together several **assistive features** into one system, like object detection to help blind users, OCR for reading text and currency detection. On top of that, she worked on making the system run faster, **tested** the individual modules, and made sure the AI services connected perfectly with the front end.
- **Sumaika Imran** was primarily responsible for the **TTS module** and backend integration. She implemented TTS to convert server responses into speech and developed the voice command input pipeline for hands-free interaction. She integrated both components with the **Fast API** backend and contributed to **Flutter app development**, connecting front-end components to backend APIs. She reviewed existing OCR solutions, designed and implemented the full OCR pipeline covering image preprocessing, text detection, and character recognition, and She **structured the backend for OCR** to integrate via the same server.
- **Qurrat Ul Ain** was primarily responsible for the **Face Recognition** and **Object Detection modules**. She implemented an embedding-based Face Net pipeline with cosine distance similarity matching, curated and annotated the dataset, applied augmentation techniques, and trained a YOLOv8 model on 77 custom classes for assistive navigation. She also contributed to the design of the **obstacle-detecting cane**.
- **Urooj Fatima** was primarily responsible for the Pakistani **Currency Detection module** and the project report. She built a custom dataset of Pakistani currency notes, applied augmentation, and trained and evaluated a **YOLOv8 model** using precision, recall, and mAP metrics. She also compiled, formatted, and proofread the **complete report** and conducted **literature research** on currency recognition and assistive technology.

Executive Summary

The See Beyond project addresses key challenges of mobility, safety, and independence faced by individuals with visual impairments. Tasks such as detecting obstacles, reading printed material, recognizing familiar people, and navigating new environments often depend on multiple separate tools. The use of multiple tools or devices often reduces autonomy and increases daily risk. Existing solutions such as white canes and mobile applications provide limited support and lack complete real-time awareness. This project responds by developing an integrated assistive platform that combines machine learning, and a companion mobile application to deliver a unified, user-centered cane and mobile application system.

The methodology follows a coordinated design process beginning with requirement analysis and a review of existing assistive technologies. The hardware workflow focuses on a cane with distance sensor, and power management circuitry. The software workflow includes the development of a Flutter-based mobile application together with lightweight real-time object and object detection models, facial recognition module, OCR-based text extraction, Pakistani currency classification, and a voice-controlled interaction interface. The mobile application also enables secure updating of user-specific recognition data. Development emphasizes testing, performance optimization, and refinement.

The results indicate Object and obstacle detections enhance safe mobility, facial recognition improves social interaction, and OCR and currency classification support literacy and financial independence. Prototype testing across indoor and outdoor environments demonstrates reliability and improvements in user confidence and independence. The project concludes that an integrated AI-powered assistive platform can improve the quality of life of visually impaired users by combining multiple essential capabilities in a single device. See Beyond demonstrates that affordable technology can address limitations of conventional tools and provides a scalable foundation for future assistive solutions.

Acknowledgments

First and foremost, we express our deepest gratitude to **Allah Almighty** for His countless blessings, strength, guidance, and mercy throughout the completion of this project. Without His help, this achievement would not have been possible.

We are profoundly thankful to our respected project supervisor, **Dr. Urooj Ainuddin**, for her invaluable guidance, continuous encouragement, constructive feedback, and unwavering support throughout this journey. Her expertise and mentorship played a vital role in the successful completion of this project.

We would also like to extend our sincere appreciation to the **Department of Computer and Information Systems Engineering** at NED University of Engineering and Technology for providing the necessary resources, facilities, and a conducive academic environment for conducting this research.

Finally, we are deeply grateful to our parents and families for their endless love, prayers, patience, sacrifices, and constant encouragement. Their support and belief in our abilities have been a source of motivation throughout our academic journey.

We acknowledge with gratitude everyone who contributed, directly or indirectly, to the successful completion of this project.

Table of Contents

Author's Declaration	ii
Statement of Contributions	iii
Executive Summary	iv
Acknowledgments.....	v
Table of Contents	vii
List of Figures	xi
List of Tables.....	xi
List of Abbreviations.....	xiii
United Nations Sustainable Development Goals	xiv
Similarity Index Report.....	xv
Chapter 1: Introduction	1
1.1 Background Information	1
1.2 Significance and Motivation.....	1
1.3 Aims and Objectives.....	2
1.3.1 Improve Situational Awareness.....	2
1.3.2 Facilitate Natural and Accessible Interaction	2
1.3.3 Promote Independence in Daily Tasks	2
1.3.4 Enhance Personal Safety and Mobility.....	3
1.3.5 Ensure Accessibility Through Cost Efficient and Energy Optimized Design.....	3
1.3.6 Support Secure and Scalable Data Management.....	3
1.3.7 Enable Seamless Integration with a Mobile Application	3
1.3.8 Maintain Reliability Through Iterative Testing and User Centered Refinement	4
1.4 Methodology	4
1.4.1 Requirement Analysis and Literature Review.....	4
1.4.2 Hardware Design and Selection	5
1.4.3 Software Development and Machine Learning Models	5
1.4.4 Mobile Application Development	5
1.4.5 System Integration and Testing.....	6
1.4.6 Optimization, Validation, and Documentation.....	6
1.5 Report Outline	7
1.5.1 Project Scope:.....	7
1.5.2 Complete Breakdown of Chapters.....	7
Chapter 2: Literature Review	11
2.1 Introduction	11
2.2 Sensor-Based Smart Canes and Electronic Mobility Aids	12
2.2.1 Architecture and Working Principles of Existing Smart Cane Systems.....	12
2.2.2 Embedded Sensors and Electronic Components Used in Smart Mobility Aids.....	13

2.2.3 Multimodal Feedback Mechanisms for User Interaction	14
2.3 Vision-Based Wearable Assistive Systems	15
2.4 Object Recognition and Obstacle Detection Techniques	16
2.4.1 YOLO-Based Object Detection.....	16
2.5 OCR-Based Text Recognition for Visually Impaired Users	17
2.5.1 OCR Processing Pipeline and Core Components.....	17
2.5.2 OCR Techniques in Assistive Vision Systems.....	18
2.5.3 OCR and Text-to-Speech (TTS) Integration	19
2.6 Currency Detection and Recognition for Visually Impaired Users.....	19
2.6.1 Traditional Currency Recognition Approaches.....	19
2.6.2 CNN-Based Currency Classification.....	20
2.6.3 YOLO-Based Currency Detection and Recognition	20
2.6.4 Challenges in Real-World Currency Recognition	20
2.6.5 Integration of Currency Recognition with Audio Feedback	21
2.7 Face Recognition Techniques for Assistive Applications	21
2.7.1 Image-Based Face Recognition.....	21
2.7.2 One-Shot Learning and Siamese Neural Networks.....	22
2.7.3 Video-Based Face Recognition and Neural Aggregation Network.....	24
2.7.4 Neural Aggregation Network (NAN).....	24
2.7.5 Summary	26
2.8 Microsoft Seeing AI	27
2.8.1 NavCog3.....	28
2.8.2 Indoor Navigation Techniques	29
2.8.3 Outdoor Navigation and Routing Technologies.....	30
2.9 Summary and Research Gaps	31
Chapter 3 Object Detection and Recognition Module	32
3.1 Introduction	32
3.2 System Overview and Functional Architecture.....	32
3.3 Limitations of Pretrained Models and Motivation for a Custom Dataset.....	33
3.4 Custom Dataset Development Using Roboflow.....	34
3.5 Data Augmentation.....	35
3.6 Annotation and Preprocessing.....	35
3.7 Annotation Conversion to YOLO Format	36
3.8 Model Fine-Tuning.....	36
3.8.1 Results	36
3.9 Limitations.....	37
3.10 Future Work.....	38

3.11 Conclusion.....	38
Chapter 4: Currency Detection Module	39
4.1 Introduction & Motivation	39
4.2 Early Approaches and Model Evolution	39
4.2.1 First Approach: Pre-Collected Dataset with YOLOv8n.....	40
4.3 Current Solution: Custom Dataset with YOLOv8s	41
4.3.1 Custom Dataset Creation and Preprocessing.....	41
4.3.2 Training Results and Performance Evaluation	43
4.4 Over-fitting and Under-fitting Analysis	46
4.5 Conclusion.....	47
Chapter 5: Optical Character Recognition	48
5.1 Introduction	48
5.2 OCR System Requirements.....	48
5.2.1 Image Pre-Processing and Normalization Framework.....	49
5.3 Mobile ML-Based OCR using Google ML Kit.....	49
5.3.1 Limitations Observed	49
5.4 Deep Learning–Based Text Detection using YOLOv8	49
5.4.1 Approach	50
5.4.2 Observations.....	50
5.5 Final OCR Approach: PaddleOCR.....	50
5.5.1 PaddleOCR Architecture Overview	51
5.5.2 Reasons for Selecting PaddleOCR	51
5.5.3 Experimental Observations	51
5.5.4 Limitations of PaddleOCR	53
5.5.5 Trade-off Analysis and Final Decision.....	53
5.6 Summary	53
Chapter: 6 People Recognition.....	54
6.1 Introduction	54
6.2 Overview	54
6.3 Working of the People Recognition Module.....	55
6.4 Previous Implementation.....	55
6.5 Challenges	56
6.6 Advantages of the Current Approach	57
6.7 Transition to High-Performance FaceNet Architecture.....	57
6.8 Methodology	58
6.9 Detailed FaceNet Workflow	59
6.10 Experimental Observations and Advantages of FaceNet	61

6.11 Conclusion.....	61
Chapter 7: Smart Blind Cane with Dual Ultrasonic Sensors and Haptic Feedback.....	62
7.1 Introduction	62
7.2 Problem Statement.....	62
7.3 Objectives	62
7.4 Hardware Components	63
7.5 Methodology	64
7.5.1 Obstacle Detection	64
7.5.2 No Obstacle Condition	65
7.6 Circuit Design.....	65
7.7 Limitations.....	66
7.8 Future Improvements.....	66
7.8 Conclusion.....	66
Chapter 8 App Development.....	67
8.1 Introduction	67
8.2 Technology Stack Rationale.....	67
8.3 Backend Architectural Design.....	68
8.3.1 Architectural Pipeline Execution.....	68
8.4 Project Structure and Modularity	69
8.5 Centralized Configuration Management	69
8.6 Service Dependency Injection Container	70
8.6.1 Cold Boot State and Lifecycle Management.....	70
8.7 Multi-Stage Face Recognition Workspace	70
8.7.1 Subsystem Pipeline Mechanics	70
8.8 Optical Character Recognition Module.....	71
8.8.1 OCR Processing Stages	71
8.9 Custom Currency Detection System.....	71
8.9.1 Model Tuning and Currency Classification	71
8.10 Blind Assistance and Threat Prioritization.....	72
8.10.1 Object Segmentation and Dynamic Priority Parsing.....	72
8.11 REST API Layer Implementation	72
8.12 Error Resiliency Framework	72
8.13 Computational Performance Optimizations	73
8.14 Security Implementations.....	73
8.15 Summary	74
Chapter 9 Frontend Development	75
9.1 Introduction:	75

9.2 Voice Input: Speech-to-Text (STT).....	75
9.2.1 Wake Word Listening	75
9.2.2 Command Listening	76
9.2.3 The Voice Loop.....	76
9.3 Background Operation:	76
9.4 Camera Capture Module.....	77
9.4.1 Foreground Capture.....	77
9.4.2 Background / Locked-Screen Capture.....	78
9.5 Intent Detection	78
9.6 Backend Communication	79
9.7 Text-to-Speech (TTS).....	79
9.8 End-to-End Pipeline	80
9.9 Module Walkthroughs	81
9.9.1 Read Text (OCR).....	81
9.9.2 Object Detection.....	81
9.9.3 Currency Detection.....	81
9.9.4 Face Recognition and Enrolment	81
9.9.5 Scene Description.....	82
9.10 Conclusion:.....	82
Chapter 10 Conclusions.....	83
10.1 Summary	83
10.2 Recommendations for Future Work	83
References	85

List of Figures

Figure 1: Sketch of the arrangement of the sensors and object detection range (Ahmed et al., 2018).....	13
Figure 2 A simple 2 hidden layer network for binary classification with logistic prediction ...	22
Figure 3 : Convolutional architecture selected for the verification task.....	23
Figure 4 : NAN architecture for video face recognition.....	26
Figure 5 : An overview of NavCog3 (Sato et al., 2017).....	29
Figure 6 : System Overview and Functional Architecture	33
Figure 7 : Comparison of training and validation loss curves.	37
Figure 8 : Data augmentation examples showing rotation, brightness variation, blur, and noise applied to currency notes.	42
Figure 9 : Training and validation loss curves showing stable convergence and absence of overfitting and Mean Average Precision (mAP@50 and mAP@50–95) curves across training epochs.....	44
Figure 10 : Confusion matrix illustrating classification performance across all currency denominations.....	45
Figure 11 : Final detection results showing bounding boxes, class labels, and confidence scores for different currency notes.	45
Figure 12 : Comparison of training and validation loss curves demonstrating balanced learning behavior.	46
Figure 13 : Sample scene-text image captured in a real-world environment and processed using the PaddleOCR pipeline.....	52
Figure 14 : Operational pipeline of the FaceNet face recognition system from preprocessing to verification.....	59
Figure 15 : Circuit Design of the Smart cane	65

List of Tables

Table 1 Quantitative evaluation metrics of the proposed model on the 77-class validation dataset	37
Table 2 Performance Metrics of the YOLOv8n Model Trained on the Pre-Collected Pakistani Currency Dataset.....	40
Table 3 Performance Metrics of the Final YOLOv8s Model Trained on the Custom Augmented Pakistani Currency Dataset	44
Table 4 Sample Text Recognition Results and Corresponding Confidence Scores Obtained Using PaddleOCR.....	52

List of Abbreviations

AI	Artificial Intelligence
BLE	Bluetooth Low Energy
CNN	Convolutional Neural Network
GPS	Global Positioning System
IMU	Inertial Measurement Unit
OCR	Optical Character Recognition
SLAM	Simultaneous Localization and Mapping
TTS	Text-to-Speech
VIBP	Visually Impaired and Blind Persons
YOLO	You Only Look Once

United Nations Sustainable Development Goals

The Sustainable Development Goals (SDGs) are the blueprint to achieve a better and more sustainable future for all. They address the global challenges we face, including poverty, inequality, climate change, environmental degradation, peace and justice. There is a total of 17 SDGs as mentioned below. Check the appropriate SDGs related to the project.

- ☐ No Poverty
- ☐ Zero Hunger
- ☒ Good Health and Well being
- ☐ Quality Education
- ☐ Gender Equality
- ☐ Clean Water and Sanitation
- ☐ Affordable and Clean Energy
- ☐ Decent Work and Economic Growth
- ☒ Industry, Innovation and Infrastructure
- ☒ Reduced Inequalities
- ☐ Sustainable Cities and Communities
- ☐ Responsible Consumption and Production
- ☐ Climate Action
- ☐ Life Below Water
- ☐ Life on Land
- ☐ Peace and Justice and Strong Institutions
- ☐ Partnerships to Achieve the Goals

Similarity Index Report

Following students have compiled the final year report on the topic given below for partial fulfillment of the requirement for Bachelor's degree in Computer and Information System Engineering.

Project Title: See Beyond: Guided by sound empowered by touch

S. No.	Student Name	Seat Number
1.	Sheikh Malaika Mustafa	CS-22006
2.	Sumaika Imran	CS-22009
3.	Qurrat UIAin	CS-22018
4.	Urooj Fatima	CS-22022

This is to certify that Plagiarism test was conducted on complete report, and overall similarity index was found to be less than 20%, with maximum 5% from single source, as required.

Signature and Date

.....

Dr. Urooj Ainuddin

Turnitin Originality Report

Processed on: 16-Jun-2026 3:57 PM PKT
 ID: 2984313800
 Word Count: 26873
 Submitted: 2

ID-11.pdf By FYP1 CIS

Similarity Index:

8%

Similarity by Source

Internet Sources: 5%
 Publications: 5%
 Student Papers: 4%

1% match (student papers from 13-Aug-2023)

[Submitted to Higher Education Commission Pakistan on 2023-08-13](#)

1% match (student papers from 28-Jul-2022)

[Submitted to Higher Education Commission Pakistan on 2022-07-28](#)

1% match (Internet from 17-Sep-2020)

<http://docplayer.net/39857991-Siamese-neural-networks-for-one-shot-image-recognition.html>

< 1% match (student papers from 26-Aug-2022)

[Submitted to Higher Education Commission Pakistan on 2022-08-26](#)

< 1% match (student papers from 19-Feb-2023)

[Submitted to Higher Education Commission Pakistan on 2023-02-19](#)

< 1% match (student papers from 15-Aug-2023)

[Submitted to Higher Education Commission Pakistan on 2023-08-15](#)

< 1% match (student papers from 13-Aug-2023)

[Submitted to Higher Education Commission Pakistan on 2023-08-13](#)

< 1% match ()

[Yang, Jiaolong, Ren, Peiran, Zhang, Dongdong, Chen, Dong, Wen, Fang, Li, Hongdong, Hua, Gang, "Neural Aggregation Network for Video Face Recognition", 2017](#)

< 1% match (Internet from 08-Mar-2026)

<https://arxiv.org/html/2505.19325v1>

< 1% match (Internet from 27-Oct-2022)

https://se.neduet.edu.pk/sites/default/files/SE/FYP/new_template/6-FYDP-Report-Guidelines.pdf

< 1% match (Internet from 08-May-2019)

<http://www.ascent-journals.com/IJERIA/Vol9No3/3-T.SYED%20AKHEEL-131.pdf>

< 1% match (publications)

[Paolo Ferro, Harinadh Vemanaboina, Chander Prakash, "Computational Techniques and Smart Manufacturing", CRC Press, 2026](#)

< 1% match (student papers from 05-May-2025)

[Submitted to The University of Manchester on 2025-05-05](#)

< 1% match (Internet from 28-Mar-2026)

http://eprints.utar.edu.my/6450/1/19ACB04197_FYP.pdf

< 1% match (Internet from 27-Mar-2026)

http://eprints.utar.edu.my/6036/1/fyp_CS_2023_LCH.pdf

< 1% match (Internet from 01-Apr-2026)

http://eprints.utar.edu.my/7560/1/fyp_CS_2025_CMK.pdf

< 1% match (Internet from 20-May-2026)

<https://dokumen.pub/computer-vision-and-machine-learning-in-agriculture-volume-3-algorithms-for-intelligent-systems-9819937531-9789819937530.html>

< 1% match (Internet from 22-Oct-2024)

<https://dokumen.pub/computer-networks-and-inventive-communication-technologies-proceedings-of-third-iccnet-2020-lecture-notes-on-data-engineering-and-communications-technologies-58-9811596468-9789811596469.html>

< 1% match (Internet from 04-Sep-2023)

<https://dokumen.pub/engineering-applications-of-neural-networks-20th-international-conference-eann-2019-xersonisos-crete-greece-may-24-26-2019-proceedings-1st-ed-978-3-030-20256-9978-3-030-20257-6.html>

< 1% match ("Technological Trends in Improved Mobility of the Visually Impaired", Springer Science and Business Media LLC, 2020)

["Technological Trends in Improved Mobility of the Visually Impaired", Springer Science and Business Media LLC, 2020](#)

< 1% match (Internet from 11-Apr-2026)

<http://dspace.daffodilvarsity.edu.bd:8080/bitstream/handle/123456789/14512/27823.pdf?isAllowed=y&sequence=1>

< 1% match (Internet from 21-Apr-2026)

<http://dspace.daffodilvarsity.edu.bd:8080/bitstream/handle/123456789/15366/P27398.pdf?isAllowed=y&sequence=1>

< 1% match (Avivit Levy, Adi Barash, Chen Zaguri, Ariel Hadad, Polina Polsky, "Improving deep learning based bluespotted ribbontail ray (*Taeniura lymma*) recognition", Ecological Informatics, 2024)

Chapter 1: Introduction

1.1 Background Information

People who are visually impaired have a time getting around safely on their own. Everyday things that people who can see take for granted like knowing what is around them not tripping over things reading signs or recognizing friends are really tough. Can be dangerous. The tools we have now to help people get around do not do enough to help them understand what is going on around them. This makes it hard for impaired people to get around talk to others take care of their money and be part of society.

New technology like intelligence, special computer systems and mobile phones can help solve these problems. Devices that can see and understand what is going on and then tell the user, about it can really help people who're visually impaired. The See Beyond project is working on this by making a device that can detect obstacles read text and money recognize faces and talk to the user. The See Beyond project wants to make something that people can really use in their lives something that is affordable and helps people who are visually impaired be safe, independent and part of their communities.

1.2 Significance and Motivation

The See Beyond project is important because it helps people with impairments have a better life. This is especially true in areas where people do not have access to tools that can help them. When people with impairments have trouble moving around, they often lose confidence and have to rely on others to help them.

The See Beyond system is special because it helps impairments to navigate safely and make good decisions when they do everyday tasks.

We started the See Beyond project because we saw a problem with the technology that is available now. Many tools only do one thing like help people navigate or read text. The See Beyond project wants to create a tool that does everything people with visual impairments need. We want to make a device that is easy to use and does not cost too much. This device will work with an application that caregivers can use to help the people they care for. The See Beyond system is designed to be affordable, easy to use and reliable so more people can have access to tools that can help them.

1.3 Aims and Objectives

1.3.1 Improve Situational Awareness

The system needs to tell people about things that're in their way like obstacles and objects and people right away. It also needs to tell them about things they see like printed text and what is going on around them so they can be safe. Feel good about moving around. This thing should work inside and outside and help people get where they are going by telling them what they need to know at the right time using audio alerts that come at the right moment. The system should provide information, about obstacles and objects and people and environmental cues to help people move around with confidence.

1.3.2 Facilitate Natural and Accessible Interaction

We want to make it easy for people to talk to devices using their voice. People should be able to say what they want and get a clear answer back. We should use voice commands and audio feedback to help people know what is going on. We also need to make sure the device is easy to use for people who're not good with technology. At the time the device should be able to do more things for people who are good with technology and want to use more complicated voice commands. The voice commands and audio feedback should be simple and intuitive so people can use the device without getting confused. We are talking about voice commands and audio feedback, for human device interaction. Human device interaction should be easy to use.

1.3.3 Promote Independence in Daily Tasks

The system is supposed to help with things like reading printed text identifying Pakistani currency and detecting signage. It is also supposed to recognize individuals. This will help people do things on their own without needing help from others. This means Pakistani currency and signage and printed text will be easier to deal with. The system will make people more independent, in their life, social life. When it comes to money.

1.3.4 Enhance Personal Safety and Mobility

We need to tell people about things that're in their way. This can be something like a hole in the ground or a wall that is not usually there. We also need to warn them about things that can hurt them. Our goal is to help people avoid getting hurt. We want to help them know what is around them so they can move around safely. This is especially important in places, like markets and streets where there are a lot of people. The System will help people navigate these areas like markets, streets and crowded areas.

1.3.5 Ensure Accessibility Through Cost Efficient and Energy Optimized Design

We need to build a system that does not use a lot of power. This system should be made with parts that're easy to find and not too expensive. The device must keep working in the real world and it has to be useful for communities that do not have a lot of technology. We are talking about an assistive system which has to be simple and easy to use, and be able to do its job without any problems.

We need to build a system that does not use a lot of power. This system should be made with parts that're easy to find and not too expensive. The device must keep working in the real world and it has to be useful for communities that do not have a lot of technology. We are talking about an assistive system which has to be simple and easy to use, and be able to do its job without any problems.

1.3.6 Support Secure and Scalable Data Management

We need to create a system that lets people we trust update information, about our faces and other details. This system has to keep our information private and work when we are connected to the internet. The facial recognition system must be safe. Allow us to add new features later on without putting our information at risk. We have to make sure that the people taking care of this system are trustworthy and that they can update the user facial recognition information safely.

1.3.7 Enable Seamless Integration with a Mobile Application

We need to make a companion application that helps with setting up devices, secure login updating the database and customizing the system. This mobile application should be easy to use for caregivers and make sure that the main functions of the device still

work properly without internet. The mobile application must support secure login, database updates and system customization so caregivers can use it easily.

1.3.8 Maintain Reliability Through Iterative Testing and User Centered Refinement

We need to do testing all the time in situations and get feedback from real users to make the system work better be easier to use and give more accurate results. The main goal of the system is to make sure it works consistently is very reliable. Meets the needs of visually impaired users. We want to make sure the system is good, for impaired users and does what they need it to do.

1.4 Methodology

The See Beyond project uses an approach to develop things. It is a step-by-step process that includes designing hardware and software using machine learning to make systems and creating mobile apps. The See Beyond project does this to make sure that what they make is easy to use works and is something that lots of people can use, including people who are visually impaired. The See Beyond project has steps. Each step of the See Beyond project helps make the system, for visually impaired individuals a little better.

1.4.1 Requirement Analysis and Literature Review

The project is going to start with a look at what we need to do. We will read a lot of books and papers to get an understanding of what has been done before. We want to see how other people have made things to help people who cannot see. We will look at things like computers that can detect objects read text recognize money, faces and voices.

We will also look at reports from experts and open-source projects to learn from them. This will help us figure out what people need what could go wrong and what we need to make sure the system can do. We will talk to people who might use this system and experts in the field to get their ideas. We want to make sure we understand what the system needs to do and how it should work. We are particularly concerned, about making sure it does not use much power, is cheap and responds quickly to the user. The project will focus on object detection, OCR, currency recognition, facial recognition and voice-enabled assistive systems to make this happen.

1.4.2 Hardware Design and Selection

In the hardware design phase we will pick the parts to help with real-time processing and save energy at the same time. We will also add parts like camera modules, distance sensors, audio output devices and power management units. These parts will help the system find obstacles and give feedback through sound or touch. When we design the hardware we will make sure it is small strong and easy to use. This way the ESP32 microcontroller and the whole system will be practical, for people who're visually impaired to use every day.

1.4.3 Software Development and Machine Learning Models

Software development is going to be done in a simple and step by step way. We will make machine learning models that can be used right away to help with tasks that need to be done quickly. We will use these models to detect objects and obstacles using techniques like YOLO-based models and Face Net-based detectors.

We will also make a system to recognize faces by comparing pictures with a database that we keep on our system so we can keep things private and reliable. We will use a tool called Paddle OCR to help us read text in languages. We will use a special model to recognize Pakistani money. We will train our models using data that's available to everyone and some data that we collect ourselves so we can make sure they work well in real life. We will test how well they work by looking at things like how accurate they're how long they take to work and how much memory they use so we can make sure they work well on small devices. We will choose models that're small and work quickly. We will also add a way for people to talk to the system and get answers back using a text-, to-speech system so people can use it easily. Get help right away.

1.4.4 Mobile Application Development

We are going to make an app that works with the system. This mobile app is going to be made using Flutter so it can work on different devices. The mobile app will be safe. Only let people who are supposed to use it like caregivers or family members get in and manage things like faces the system knows and information, about those faces.

The mobile app will also let people set up their device and sync the database and customize the system. The device and the mobile app will talk to each other using wireless like Wi-Fi or Bluetooth. This way the mobile app can work even when it is not connected to the internet. The data will be safe. If we need, we can also use the cloud to update some things. Only if it is not important and it will not hurt the privacy and reliability of the system.

1.4.5 System Integration and Testing

Once all system components are developed, they will be integrated into a unified mobile application through step-by-step assembly to identify integration issues early. Testing will be conducted at multiple levels, including individual modules, system integration, and real-world usage scenarios, along with user feedback to evaluate usability, accuracy, and reliability. By the midpoint of the project, a functional prototype will be available, supporting core features such as object detection and face recognition, with initial integration into the mobile application.

1.4.6 Optimization, Validation, and Documentation

The final phase of our approach will focus on making the system run smoothly and work well. We will work on reducing energy use making processing and getting response times right. This will help ensure the system operates steadily in situations. We will do testing with real users to check if the system works well at both component and system levels. We will keep user data safe and private by storing it on the device and using access controls. This way sensitive data stays protected and the system We will create guides, including how the system is built, how to use it and how to set it up. These guides will help with putting the system in place and making it better, in the future.

1.5 Report Outline

1.5.1 Project Scope:

The "See Beyond" project focuses on the design, complete development, and thorough assessment of an economical, low-latency, and multi-modular support system specifically created for people with visual impairments. The technical scope effortlessly connects embedded hardware design with edge-centric computer vision systems.

Hardware Infrastructure: The design of an ergonomic, durable, and lightweight smart cane featuring a coordinated dual-ultrasonic sensor setup. This subsystem is designed to dynamically detect spatial limits, recognizing high-level hazards (like low-hanging obstacles and face-height projections) and low-level ground barriers, converting sensor information into multi-level haptic feedback (vibrational patterns) managed by an Arduino microcontroller.

On-Device Computer Vision Framework: The development, refinement, and local implementation of extensively optimized deep learning components. This encompasses 77-class real-world object detection, tailored classification of Pakistani currency under unfavorable lighting conditions, smart text localization and extraction (OCR), and a local face verification system employing metric learning.

Cross-Platform Mobile Integration: The creation of an easy-to-use, voice-activated mobile application utilizing the Flutter framework that securely links to a streamlined, parallel-processing FastAPI backend server. The application serves as the main sensory interface for users, employing native text-to-speech (TTS) and speech-to-text (STT) loops to translate user intents into visual analysis without needing intricate tactile engagement.

1.5.2 Complete Breakdown of Chapters

The following chapters of this report record the ongoing lifecycle of the project, from idea generation and practical research to detailed implementation and practical assessment.

Chapter 2 offers a comprehensive overview of the historical and current advancements in Electronic Mobility Aids (EMAs) and computer vision frameworks. It investigates the evolutionary development of real-time object detection models (specifically contrasting different YOLO framework variants), outlines the processes involved in scene-text extraction, and compares embedding-based face verification with conventional image classification. Additionally, it examines existing assistive technologies like Microsoft Seeing AI and Orcam MyEye, emphasizing significant shortcomings—such as exclusion of regional currencies, absence of hardware integration, and elevated computational latency which directly influence the design of "See Beyond."

Chapter 3 focuses on creating the core environment-mapping system. It describes the approaches used in creating a tailored 77-class dataset with Roboflow to focus on essential everyday items. It addresses challenges in data collection, standard annotation guidelines for bounding boxes, and sophisticated image augmentation methods (including geometric scaling, exposure modifications, and noise addition). Ultimately, it details the training hyperparameters, the architectural framework, and the fine-tuning methods applied to the YOLOv8 model, concluding with an extensive performance evaluation using Mean Average Precision.

Chapter 4 aims to address a significant issue encountered by the local blind community—the precise identification of local paper money. This chapter examines the continuous design developments of a specialized classification network. It records the shift from initial exploratory model weights to a tailored YOLOv8 small (YOLOv8s) structure. The chapter discusses localized datasets, fine-tuning setups, and practical testing conditions aimed at tackling structural deformation, rips, creases, and significant fluctuations in lighting conditions while identifying Pakistani banknotes.

Chapter 5 records the practical choice and execution of a strong, edge-compatible text reading module. It details the technical assessment of various frameworks, beginning with fundamental API tests using Google ML Kit to tailored custom YOLO-based text localization arrays. It offers an in-depth examination of the completed pipeline established on the PaddleOCR framework that allows users to effortlessly read signs, documents, and product labels.

Chapter 6 outlines the development of a customized social aid system utilizing secure facial recognition. It offers a detailed examination of the architectural transition from resource-heavy classification networks to metric-learning approaches. It outlines the execution of a FaceNet deep learning structure that produces low-dimensional vector embeddings from facial crops. It details the incorporation of mathematical distance assessment frameworks (like Cosine Similarity and Euclidean Distance) to manage face verification, personalized database registration, and real-time user tagging with maximum accuracy.

Chapter 7 outlines the physical design, electronic diagrams, and embedded system algorithms that control the smart cane prototype. It explains the design reasoning for using an Arduino Nano microcontroller because of its minimal power usage and small size. It specifies the geometric arrangement of two ultrasonic sensors to remove blind spots, the mathematical adjustment of time-of-flight distance measurements, and the firmware state machine developed to control a variable haptic vibration motor to proactively notify users of impending spatial dangers.

Chapter 8 examines the foundational structure of the software ecosystem, concentrating on the robust backend architecture. It outlines the implementation of a FastAPI framework noted for its asynchronous processing endpoints. It outlines the structural flow of RESTful APIs, modular dependency injection methods, and the execution contexts used to compile and load variations of TFLite or ONNX models. Particular focus is placed on optimization methods used to manage simultaneous API inference requests from the mobile application while ensuring minimal computational latency.

Chapter 9 outlines the development of the user-focused interface built using the Flutter cross-platform framework. It investigates the accessibility tools created for those with visual impairments, such as built-in options for personalized Text-to-Speech (TTS) audio prompts and an automated local Speech-to-Text (STT) system for voice commands. It also emphasizes intricate mobile background threading mechanisms, illustrating how the app sustains ongoing background execution, oversees a locked-screen camera pipeline, and transmits live video feeds straight to the backend processing layer.

Chapter 10 is concluding section which consolidates the collective results, performance indicators, and social effects of the unified "See Beyond" ecosystem. It details the main technical milestones across all modules, tackles recognized operational limitations, and presents a strategic plan for upcoming improvements, including transitioning to entirely on-device Edge-AI compilation, enhancing power consumption profiles, and establishing cloud-based multi-user synchronization.

Chapter 2: Literature Review

2.1 Introduction

The World Health Organization says that a lot of people around the world have trouble seeing. In fact, more than 2.2 billion people have some kind of vision problem like low vision or complete blindness. This makes it hard for people who're visually impaired and blind to do everyday things on their own like getting around new places recognizing people reading or handling money. They often need help from others, which can make them feel less independent and less confident. There are some tools that can help like white canes and guide dogs. These tools are good for giving feedback about things that're close by but they are not very good at giving information about what is around them like what things are or who people are. They are also pretty cheap and reliable. They have some big limits. For example, they cannot tell people who is standing in front of them or what a sign says.

Recently computers have gotten a lot better at understanding what they see. This has led to the development of new tools that can help people who are visually impaired and blind. These tools can do more than just detect obstacles they can actually understand what is around them. (Tapu et al., 2020)

This part of the book looks at all the tools that are already out there to help people who are visually impaired and blind. We look at tools that use cameras, devices that you wear and special computers that can recognize objects and people. We also look at tools that can read text out loud and help people get around inside and outside. We compare all these tools to see what they are good at and what they are not good at and to find out what still needs to be worked on. This will help us understand what is missing and how we can make something that is better. The World Health Organization and people, like Tapu have done a lot of work in this area. We can learn from them.

2.2 Sensor-Based Smart Canes and Electronic Mobility Aids

2.2.1 Architecture and Working Principles of Existing Smart Cane Systems

Smart canes are electronic mobility aids that incorporate sensors, microcontrollers, and feedback systems to increase the usefulness of conventional white canes. These devices are intended to identify impediments that are out of the reach of a traditional cane and notify the user in real time, so that the user may be confident about the near surrounding. The Guide Cane, created at the University of Michigan, is among the first and most significant designs. Ulrich and Borenstein (1997). The Guide Cane is made up of a sensor head that is fixed to a two-wheel axle at the tip of the cane. An onboard control unit calculates safe travel routes while ultrasonic sensors continuously examine the surroundings for obstacles. In contrast to passive canes, the Guide Cane actively navigates around obstructions by sending physical force feedback through the handle, enabling the user to follow a safe path. A multi-sensor smart cane that combines infrared (IR) and ultrasonic sensors for obstacle identification at various heights was proposed by Ahmad et al. (Ahmad et al., 2018) While the infrared sensor concentrates on surface discontinuities like stairs, potholes, and lowering slopes, the ultrasonic sensors identify impediments from ground level up to head height. After processing sensor data, the microcontroller activates audio alarms and vibration motors. Despite being affordable, the device has issues with noise sensitivity and hand motion-induced sensor misalignment.

NavCane, a complete assistive technology that combines ultrasonic detection, RFID identification, gyroscopic orientation tracking, GPS, and GSM modules, was presented by (Meshram et al., 2019). Obstacle detection, wet surface recognition, object tagging, and emergency location sharing are all made possible by the system architecture. However, these multi-module systems' complexity and power consumption continue to be significant drawbacks.

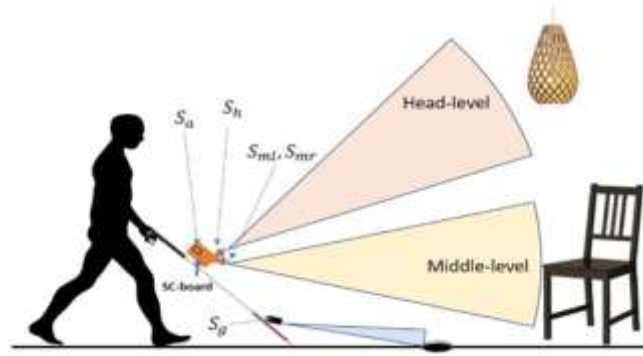


Figure 1: Sketch of the arrangement of the sensors and object detection range (Ahmed et al., 2018)

Smart cane technology is practically used in commercial systems like WeWALK and UltraCane. While UltraCane mainly concentrates on haptic obstacle feedback, WeWALK incorporates ultrasonic sensors, inertial measurement units (IMUs), Bluetooth connectivity, and smartphone navigation.

2.2.2 Embedded Sensors and Electronic Components Used in Smart Mobility Aids

For accurate environmental awareness, smart canes use a variety of sensing technologies:

- **Ultrasonic Sensors:**

These sensors operate by emitting ultrasonic waves and measuring the echo return time to calculate distance. They typically provide a detection range of **2–4 meters** and are widely used due to low cost and simplicity .

- **Infrared (IR) Sensors:**

IR sensors are useful for short-range obstacle detection (1-2 meters) since they can detect reflected infrared radiation. However, their outside usability is limited due to their sensitivity to sunshine and surface reflectance.

- **LiDAR Sensors:**

LiDAR systems use time-of-flight measurements and laser pulses to create high-resolution 3D maps. Although they are precise, their high cost and high power consumption make them inappropriate for low-cost cane designs.

- **Inertial Measurement Units (IMUs):**

By monitoring the cane's orientation, motion, and tilt, accelerometers and gyroscopes enhance stability and context awareness.

- **GPS Modules:**

Outdoor positioning and route direction are made possible via GPS, however indoor signal attenuation prevents it from working.

- **Wireless Communication Modules:**

Cloud processing and user customisation are made possible by Bluetooth and Wi-Fi modules that provide data interchange between the cane and mobile applications. (Borenstein & Ulrich, 1997; Ahmad et al., 2018)

2.2.3 Multimodal Feedback Mechanisms for User Interaction

Since VIBP users cannot rely on visual cues, smart assistive devices employ **non-visual feedback channels**:

- **Haptic Feedback:** Vibration motors produce tactile alerts that show the direction and proximity of obstacles in different patterns or intensities. This technique works well in noisy settings and is quick and covert.
- **Audio Feedback:** Beeps and text-to-speech (TTS) spoken instructions are examples of audio cues that include comprehensive semantic information such item kind and navigation instructions. On the other hand, ambient noise may be disrupted by audio feedback. In order to maximize usability in a variety of settings, the majority of

contemporary systems use a multimodal feedback method that combines haptic and aural inputs.(Ghodake et al., 2023)

2.3 Vision-Based Wearable Assistive Systems

Recent studies have found that wearable devices like glasses that you can see with are really helpful for people who are blind or cannot see very well. These devices are better than the tools that people used to get around. Asha Ghodake and her team made a pair of smart glasses that can recognize faces. This helps people who're blind to move around on their own and know what is going on around them.

The system they made uses sound waves to detect things a computer to see things and audio to tell the person what is happening. It can find things in the way objects and even peoples faces in time. They used a Raspberry Pi to make it all work together. This little computer is connected to a camera that takes videos and a special sensor that can tell how away things are.

The sensor sends out waves and gets them back to figure out how far away something is. This helps the person know if something is in their way. At the time the camera is taking pictures and using a special tool called YOLO to find many things in one picture. YOLO is good because it works fast and can find things correctly. This is very important, for devices that people wear to help them. (Ghodake et al., 2023)

The system has a way to recognize peoples faces. It keeps a collection of pictures of people it knows. When the camera sees someone it checks if that person is in the picture collection. If they are it says who that person is. If not it says the person is unknown. This is really helpful for people who're blind. They can hear who is around them. It helps them talk to people. The system tells them who is there using a computer voice that they can hear in their earphones. They do not need to use their hands to hear this information.

The system can also read things out loud that are written. It uses a tool called Pytesseract to do this. When it takes a picture of something with writing it makes the picture clearer so it can read the writing. Then it uses a computer voice to tell the person what the writing says. This is really helpful because it can read signs,

labels and other things that are written. It helps people know what is, around them. It makes it easier for them to get around.

2.4 Object Recognition and Obstacle Detection Techniques

Assistive navigation and environmental awareness systems are based on object recognition and obstacle detection. Early methods used classical classifiers in conjunction with manually created features like SIFT, SURF, and HOG. However, because to fluctuations in viewpoint, occlusion, and illumination, these methods performed poorly in situations. By enabling automatic feature learning from data, deep Convolutional Neural Networks (CNNs) transformed object detection. CNN-based detectors extract hierarchical features, where early layers capture edges and textures, while deeper layers learn semantic representations of objects.

2.4.1 YOLO-Based Object Detection

A single-stage object detection framework called You Only Look Once (YOLO) was created for real-time performance. YOLO completes object localization and classification in a single network forward pass, in contrast to two-stage detectors. The network divides the input image into a grid and predicts bounding boxes, confidence scores, and class probabilities for each grid cell.

YOLO architectures include:

- **Backbone:** A CNN (e.g., Darknet) responsible for feature extraction.
- **Neck:** Feature aggregation layers that combine information from multiple scales.
- **Head:** Output layers that predict bounding box coordinates and class labels.

The YOLO system is really good at giving answers, which is important for systems that need to help people right away. Some studies have shown that YOLO is very good at finding things that might be in the way like people, cars, doors and furniture. This makes YOLO a good choice for systems that help people who cannot see well. (Redmon et al., 2016)

There are two types of systems that use deep learning to find objects: two-stage detectors and one-stage detectors. YOLO and SSD are examples of one-stage detectors. The other type, two-stage detectors can find objects accurately but they need a lot of computer power to do it. This means they are not good for systems that need to work in time on smaller devices. (Zhao et al., 2019)

The YOLO system, which stands for You Only Look Once is a one-stage detector. It can find objects. Tell what they are in one step using a type of neural network called a convolutional neural network. The YOLO system has a main parts: one part that looks at the picture and finds important features and another part that figures out where the objects are and what they are. There are also versions of YOLO like YOLOv5, which are even faster and use less memory making them good, for real-time use.

2.5 OCR-Based Text Recognition for Visually Impaired Users

Optical character recognition, or OCR, is a crucial part of assistive technology because it enables visually impaired persons to access printed and digital textual information found in everyday environments. OCR-based assistive technologies allow users to read books, newspapers, signboards, product labels, medication instructions, and official documents without the need for outside help. With the integration of computer vision and deep learning, OCR has evolved from simple character recognition into a potent text comprehension pipeline suitable for real-world applications.

2.5.1 OCR Processing Pipeline and Core Components

A typical OCR system follows a multi-stage pipeline consisting of the following steps:

1. Image Acquisition:

A camera installed on cellphones, goggles, or smart glasses is used to take pictures. In order to improve focus and clarity, image capture for assistive apps is frequently initiated automatically when an item is recognized within a specific distance utilizing proximity or ultrasonic sensors.

2. **Image Preprocessing:**

One crucial stage that has a big impact on OCR accuracy is preprocessing. Typical preprocessing methods consist of:

- a. Grayscale conversion to lower computational complexity
- b. Noise removal using median or Gaussian filters
- c. Removing noise with Gaussian or median filters
- d. Contrast enhancement for low-light environments
- e. Correcting perspective and deskewing text that is skewed'

3. **Text Detection:**

Connected component analysis, contour detection, or deep learning-based text detectors like EAST or CTPN are used to localize text regions. Text and non-text parts are separated in this step.

4. **Character Recognition:**

Recognized text regions are processed by OCR engines such as Tesseract, which employ LSTM-based recurrent neural networks to sequentially recognize characters. Tesseract's multilingual nature makes it suitable for multilingual assistive solutions.

5. **Post-Processing:**

Post-processing involves formatting, language modeling, and dictionary-based correction to improve recognition accuracy and readability.

2.5.2 OCR Techniques in Assistive Vision Systems

To improve situational awareness, a number of studies incorporate OCR into wearable assistive technology. OCR is implemented using Pytesseract in the smart glasses system suggested by Ghodake et al., where text extraction is preceded by thresholding and contrast enhancement of collected pictures. Hands-free reading is made possible by the TTS conversion of the extracted text into speech. In a similar vein, smart goggles have merged OCR with object detection algorithms. Their approach ensures that text is readable and reduces processing overhead by only taking pictures when close items are

spotted. Multilingual accessibility is demonstrated by further translating the OCR output into several languages utilizing translation APIs. (Ghodake et al., 2023)

2.5.3 OCR and Text-to-Speech (TTS) Integration

Recognized text needs to be turned into auditory output in order for visually impaired individuals to utilize OCR. Offline voice synthesis engines and text-to-speech (TTS) solutions like Google TTS are frequently utilized. An end-to-end pipeline is produced when OCR and TTS are integrated:

Camera → OCR → Text Processing → TTS → Audio Output

Through speakers or earbuds, this pipeline enables users to hear verbal feedback. However, OCR performance is extremely susceptible to backdrop clutter, motion blur, lighting conditions, which continue to be major obstacles in practical applications. To increase robustness, current research focuses on AI-based text identification models. (Sivaprasad et al., 2024)

2.6 Currency Detection and Recognition for Visually Impaired Users

Currency identification is an essential assistive feature that enables visually impaired individuals to independently identify banknote denominations during financial transactions. Given the high danger of fraud and dependence associated with manual currency handling, automated currency recognition systems are a crucial component of modern assistive technology.

.

2.6.1 Traditional Currency Recognition Approaches

Color histograms, edge detection, texture analysis, and template matching were among the manually created features used by early cash recognition systems. These methods' usefulness in real-world situations was limited by the need for regulated settings and constant lighting. Accuracy was greatly diminished by variations in note orientation, folding, wear and tear, and backdrop clutter.

2.6.2 CNN-Based Currency Classification

Convolutional Neural Networks (CNNs) took the lead in currency recognition with the introduction of deep learning. From labeled currency photos, CNN-based classifiers automatically pick up discriminative features including patterns, symbols, and textures.

Within these systems:

- Input images are resized and normalized
- Feature maps are extracted through convolutional layers
- Fully connected layers classify currency denominations

Under controlled circumstances, CNN-based classifiers exhibit great accuracy; nevertheless, they perform poorly when numerous notes appear in the same frame or when currency notes are partially obscured.

2.6.3 YOLO-Based Currency Detection and Recognition

YOLO-based approaches treat currency recognition as an object detection problem rather than pure classification. Instead of classifying a single cropped image, YOLO detects and classifies currency notes directly within a scene. Advantages of YOLO-based currency detection include:

- Simultaneous localization and denomination recognition
- Robust performance under partial occlusion
- Ability to detect multiple currency notes in one frame
- Real-time inference suitable for assistive systems

YOLO-based currency detection is particularly suitable for wearable assistive devices and mobile applications due to its fast inference speed and relatively small model size. (Redmon et al., 2016)

2.6.4 Challenges in Real-World Currency Recognition

Despite advancements, several challenges remain in real-world deployment:

- Worn and damaged notes
- Folded or partially visible currency
- Illumination variations
- Background clutter
- Motion blur from handheld capture

- To guarantee dependable performance on embedded and mobile platforms, research focuses on data augmentation, illumination normalization, and lightweight model optimization.

2.6.5 Integration of Currency Recognition with Audio Feedback

In assistive systems, recognized currency denominations are converted into audio output using TTS systems. The feedback typically includes:

- Currency type (e.g., Pakistani Rupee)
- Denomination value
- Optional confidence score

This information allows visually impaired users to independently verify currency without assistance, significantly improving confidence and autonomy.

2.7 Face Recognition Techniques for Assistive Applications

Face recognition enhances social interaction, safety, and situational awareness by allowing visually challenged users to recognize familiar faces. Facial recognition must function in assistive systems despite real-world limitations such as uncontrolled lighting, motion blur, occlusions, and a small amount of enrollment data per person. Furthermore, the adoption of effective, locally deployable recognition systems is motivated by the important needs of privacy and offline operation. Depending on whether recognition is carried out using a single facial image or several frames collected over time, face recognition techniques can be generally divided into image-based and video-based approaches.

2.7.1 Image-Based Face Recognition

Deep convolutional neural networks (CNNs) are used by image-based face recognition algorithms to extract a fixed-length feature embedding from a single face image. Well-known models like FaceNet, VGG-Face, and DeepFace optimize either classification-based or metric-learning objectives to learn highly distinguishing embeddings. To verify or recognize identities without retraining the network for new users, face matching is then carried out by calculating distances between embeddings using Euclidean or cosine similarity.

These methods work good in real-time applications. They are computationally efficient. However, in unrestricted settings, where images may be impacted by changes in illumination, position fluctuations, motion blur, or partial occlusion circumstances frequently found in assistive applications, the performance decreases. Koch et al., 2015)

2.7.2 One-Shot Learning and Siamese Neural Networks

Because users may only supply a limited number of enrollment photographs, one of the main challenges in assisted face recognition systems is the limited amount of training data per individual. Conventional multi-class classification methods are inappropriate for scalable assistive applications because they necessitate large amounts of labeled data and retraining when new identities are added. In order to overcome this issue, Siamese Neural Networks reformulate face recognition as a similarity learning (verification) problem instead of explicit identification categorization.

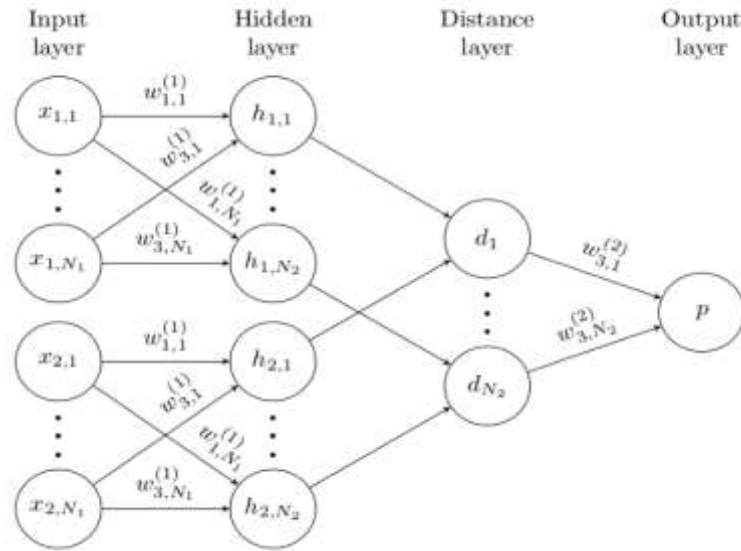


Figure 2 A simple 2 hidden layer network for binary classification with logistic prediction

The structure of the network is replicated across the top and bottom sections to form twin networks, with shared weight matrices at each layer

Twin neural network branches with shared parameters make up Siamese networks, which process two input images simultaneously and train a discriminative embedding space that pushes different faces farther away and maps similar faces closer together. A distance metric, typically the L1 or L2 distance between the feature vectors taken from the twin networks, is calculated during inference to quantify face similarity.

Using just one or a few reference images per identity, the work (Koch et al., n.d.) shows that deep convolutional Siamese architectures can successfully generalize to previously unexplored classes. Learned embeddings provide great discriminative strength and allow one-shot and few-shot recognition without retraining, even for identities not observed during training, by training the network on verification pairings instead of predefined identity labels.

Deep CNNs are usually used for feature extraction in convolutional Siamese designs intended for verification tasks, which are then followed by a distance computation layer. Specifically, it has been demonstrated that L1 component-wise distance can effectively capture small changes across high-dimensional facial embeddings prior to a final logistic similarity prediction. The fundamental idea of learning a task-independent similarity function that is applicable across domains is further demonstrated by simpler Siamese configurations, such as two-hidden-layer networks with shared weights.

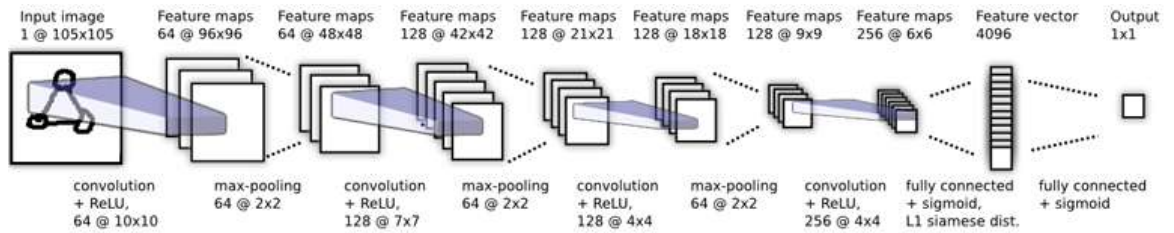


Figure 3 : Convolutional architecture selected for the verification task

This paradigm is particularly well-suited for image-based face recognition in assistive systems, where limited training samples, scalability to new users, and quick enrollment are typical needs. Nevertheless, Siamese models continue to be sensitive to input picture quality despite their excellent generalization capabilities. When faces are taken under extreme position fluctuations, motion blur, occlusion, or poor illumination conditions—all of which are common in real-world assisting scenarios—performance may suffer. These drawbacks encourage future attempts to investigate more reliable representations and aggregation techniques.

2.7.3 Video-Based Face Recognition and Neural Aggregation Network

Video-based face recognition uses several frames of the same person taken over time to provide a more reliable identity representation than image-based face recognition techniques, which rely on a single facial sample. This method works especially well in unrestricted settings where individual frames may experience blur, occlusion, position change, or low lighting—conditions commonly found in wearable and assistive technologies. Aggregating characteristics from several frames while suppressing noisy or low-quality samples is the main issue in video-based recognition.

(Koch et al., n.d.) The process of learning good features for machine learning applications can be very computationally expensive and may prove difficult in cases where little data is available. A prototypical example of this is the one-shot learning setting, in which we must correctly make predictions given only a single example of each new class. In this paper, we explore a method for learning siamese neural networks which employ a unique structure to naturally rank similarity between inputs. Once a network has been tuned, we can then capitalize on powerful discriminative features to generalize the predictive power of the network not just to new data, but to entirely new classes from unknown distributions. Using a convolutional architecture, we are able to achieve strong results which exceed those of other deep learning models with near state-of-the-art performance on one-shot classification tasks. (Koch et al., n.d.) (Yang et al., 2017).

2.7.4 Neural Aggregation Network (NAN)

For video face identification, Yang et al. presented the Neural Aggregation Network (NAN), which takes an unordered set of face photos with variable cardinality or a face video and creates a compact, fixed-dimensional representation. The feature embedding module and the aggregation module are the two primary components of the NAN framework.

Each aligned face image is mapped into a normalized low-dimensional feature vector by the feature embedding module using a deep convolutional neural network. Using a content-adaptive attention mechanism, the aggregation module then merges these feature vectors to create a single representation that falls under the convex hull of the

input features. The aggregation procedure is independent of the duration of the video and invariant to the order of incoming frames because of the attention-based design. NAN learns to automatically give greater weights to high-quality face photos while suppressing low-quality samples impacted by blur, occlusion, excessive posture, or incorrect exposure, in contrast to naive aggregation techniques like average or max pooling. Interestingly, no extra supervision signals are needed for attention learning because the network is trained using ordinary classification or verification loss functions. Experimental findings on benchmark datasets such as YouTube Face, Celebrity-1000, and IJB-A show that NAN routinely achieves state-of-the-art recognition accuracy and beats traditional aggregation approaches.

Architecture of NAN

The NAN framework consists of two main components:

1. Feature Embedding Module

A deep CNN (e.g., GoogLeNet with Batch Normalization) maps each face image into a 128-dimensional normalized feature vector. Each feature vector is given adaptive weights via an attention mechanism, which then calculates a weighted total to create a single video-level representation. NAN learns to suppress low-quality frames impacted by blur, occlusion, or extreme posture and prefer high-quality face images, in contrast to average or max pooling.

2. Attention-Based Aggregation

The aggregation module uses a learnable kernel and dot-product similarity between feature vectors to calculate attention scores. To ensure that the aggregation is order-invariant and unaffected by video length, these scores are normalized using a softmax function. By dynamically modifying attention according to the aggregated representation, a cascaded two-attention-block design improves content-aware aggregation even more.

Advantages of NAN

- Produces compact representations with constant-time matching.
- Robust to noise and frame quality variations.
- Outperforms naive pooling methods on benchmark datasets such as IJB-A, YouTube Face, and Celebrity-1000.
- Suitable for assistive applications where computational efficiency and reliability are critical.

Below is the compact representation is used for recognition.

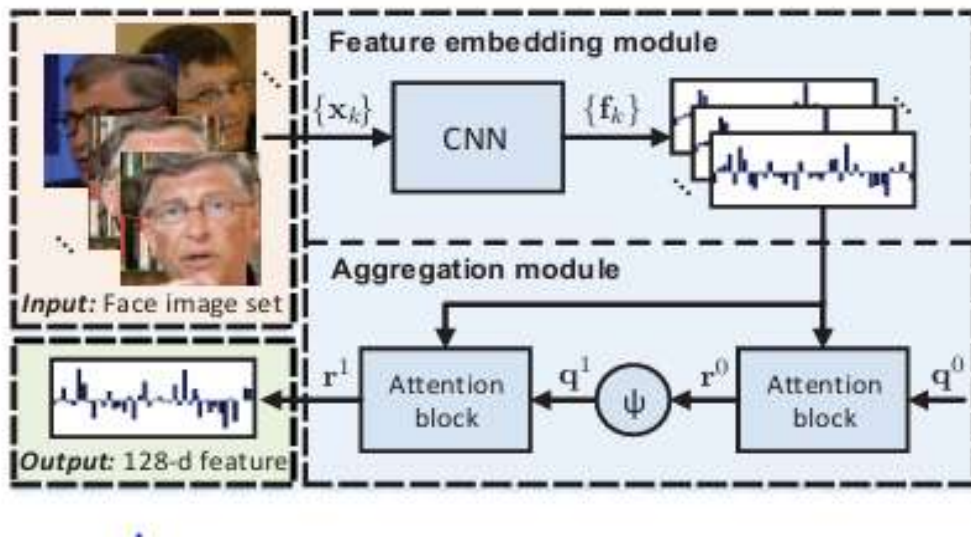


Figure 4 : NAN architecture for video face recognition.

All input face images $\{x_k\}$ are processed by a feature embedding module with a deep CNN, yielding a set of feature vectors $\{f_k\}$. These features are passed to the aggregation module, producing a single 128-dimensional vector to represent the input face.

NAN is especially well-suited for assistive applications where dependable recognition must be carried out in dynamic and uncontrolled conditions because of its compact representation, cheap computational overhead, and resilience to real-world fluctuations.

2.7.5 Summary

Previous research shows that Siamese neural networks and image-based systems is good for identifying people with little enrollment data and support both one-shot and few-shot learning scenarios. These methods are impressive for assistive devices

because they help quick enrollment and scalability without retraining. However, because they rely on single-frame inputs, they are prone to changes in illumination, position, and image quality. These constraints are addressed by video-based techniques like the Neural Aggregation Network, which improve robustness under various settings by aggregating information across numerous frames employing attention mechanisms. Even while these techniques work better, they add more computing complexity and might not be enough for all real-time assistive systems with limited resources.

Furthermore, existing assistive face recognition implementations, such as those described in (Ghodake et al., 2023) typically store raw facial images in a database and perform direct image matching or labeling. While effective for small-scale systems, this strategy incurs significant storage overhead, raises privacy concerns, and does not scale efficiently as the number of enrolled individuals increases. There is clear research cancelled for scalable, lightweight, privacy-preserving face recognition system that allows real-time recognition in order to overcome these limits. Fast similarity-based matching using cosine similarity is made possible by storing compact facial embeddings rather than raw images, which significantly lowers storage needs. This approach obeys with the practical limitations of assistive programs, protects user privacy, and enhances scalability to a greater number of known users. These findings inspired the people recognition strategy used in this project, which combines the advantages of embedding-based recognition with effective storage and real-time inference appropriate for assistive use.

2.8 Microsoft Seeing AI

Seeing AI is a widely used mobile application developed by Microsoft that demonstrates the practical application of AI for assistive vision. The app employs **deep neural networks** to perform multiple tasks:

- **Face Recognition:**

The application captures multiple images of a person and stores facial embeddings. Upon rescanning, it compares embeddings using similarity metrics to identify the individual.

- **Optical Character Recognition (OCR):**

Text is extracted from images using OCR pipelines and read aloud via TTS.

- **Scene Description:**

CNN-based image captioning models analyze visual scenes and generate descriptive sentences.

Seeing AI illustrates how **camera input** → **AI inference** → **speech output** pipelines can significantly enhance independence for visually impaired users.
(Microsoft, 2021)

2.8.1 NavCog3

NavCog3 is a smartphone-based navigation system designed for visually impaired users, with a strong focus on indoor navigation (Sato et al., 2017). Unlike basic assistive tools, it provides precise localization, turn-by-turn guidance, and environmental information, making it suitable for complex indoor spaces.

It was evaluated in large-scale environments such as a 21,000 m² shopping mall, where both controlled and real-world user studies involving visually impaired participants demonstrated its effectiveness in independent navigation. NavCog3 estimates user location using Bluetooth beacons combined with smartphone sensors like the gyroscope and accelerometer to track movement and orientation.

The system delivers step-by-step voice-based instructions and descriptions of nearby points of interest, enabling users to navigate and understand their surroundings through continuous audio feedback.

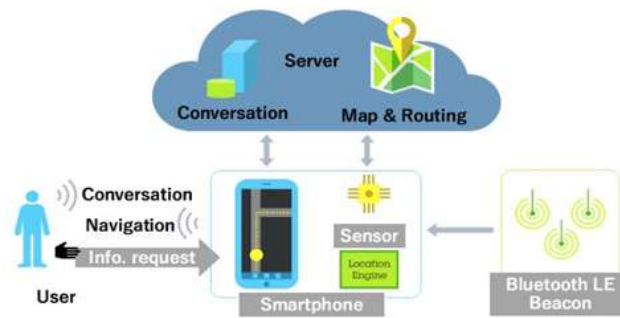


Figure 5 : An overview of NavCog3 (Sato et al., 2017)

2.8.2 Indoor Navigation Techniques

GPS is unreliable in indoor environments including schools, colleges, shopping centers, airports, and business buildings since there are frequently weak satellite signals. Numerous alternative technologies, including Bluetooth beacons, Wi-Fi location systems, RFID tags, cameras, and sensors to aid with navigation, have been developed to address this issue. Together, these technologies facilitate interior navigation.

- Bluetooth Low Energy (BLE) Beacons:** Because of its short range communication and low power consumption, BLE technology is frequently employed. These 16 F/SOP/FYDP 02/06/00 BLE beacons are positioned thoughtfully throughout a structure to send out signals on a regular basis. Every beacon transmits a universally unique identifier (UUID) that is easily picked up by a smartphone or other BLE-enabled device. This allows the user's device to monitor the Received Signal Strength Indicator (RSSI) from the beacons whenever they travel around the environment. In this manner, the RSSI values allow the system to employ fingerprinting to approximate the user's position.(Campañá et al., 2017).

- Wi-Fi Positioning Systems (WPS):** Since most buildings already have Wi-Fi networks, WPS is regarded as one of the most widely used technologies for indoor navigation. It measures the RSSI of signals it detects from neighboring Wi-Fi routers or access points. In order to determine a person's location, the system records Wi-Fi signals at several locations across the building and compares the real-time signal strengths to a pre-made map of the building called a "fingerprint map."(Ali et al., 2019).

- **Magnetic Field Mapping:** Because of metal constructions, electrical wire, and other materials, magnetic fields inside buildings can vary and can cause problems using traditional compasses. They can, however, be transformed into helpful landmarks for indoor navigation. Data regarding these variations can be gathered using the magnetometer—a sensor that measures magnetic fields—integrated into cellphones. (Gozick et al., 2011).
- **LiDAR and SLAM:** When combined with SLAM (Simultaneous localization and mapping), LiDAR is used for indoor navigation. This allows devices, such as mobile systems, to create a map of their surroundings and track their position within it. LiDAR creates a detailed 3D map of the space by emitting laser beams and timing how long it takes for them to return. (Debeunne & Vivet, 2020).

2.8.3 Outdoor Navigation and Routing Technologies

The Global Positioning System (GPS), which uses satellite trilateration to determine user location, is the main technology used for outdoor navigation. Based on road networks, routing engines like OpenStreetMap and OpenRouteService APIs determine the best routes. To increase safety, some systems also use camera-based object detection and RFID landmarks.

2.3.2 Techniques for Outdoor Navigation Although satellite-based systems are the mainstay of outdoor navigation, a number of improvements have been made to increase precision and dependability: 17 F/SOP/FYDP 02/06/00

- **Global Positioning System (GPS)** GPS is a satellite-based system that provide accurate location data. Because it improves mobility and independence, it is essential to the development of assistive equipment like smart canes for VIBP, as was previously mentioned. These signals are picked up by an integrated GPS receiver in gadgets like cellphones. Through the processing of data from several satellites, the devices are able to accurately determine their location, including latitude, longitude, and altitude, which increases the dependability and ease of navigation for users. (Ali et al., 2019)

- **RFID Technology** **Radio-Frequency Identification (RFID)** technology is a wireless sensor that relies on electromagnetic signal detection. Some ETAs employ radio frequency identification (RFID) to locate specific things, like doors, elevators, or other pre-arranged markers, to aid navigation. These objects' RFID tags interact with the devices to give users real-time information about how close they are to those landmarks.

2.9 Summary and Research Gaps

The research on vision-based assistive paradigms, smart wearables, deep learning-based object and currency identification, face recognition methods, OCR systems, mobile assistive applications, and navigation solutions for people with visual impairments was reviewed in this chapter. Significant progress was shown in the literature, but there were also drawbacks, including fragmented functionality, reliance on internet connectivity, expensive hardware, and restricted offline capability. These results emphasize the need for a comprehensive, reasonably priced, offline-capable assistive system that integrates facial recognition, object detection, OCR, currency recognition, and navigation support on a single platform. The design and development of the suggested See Beyond system are directly motivated by the highlighted research gaps. (Ghodake et al., 2023)

Chapter 3 Object Detection and Recognition Module

3.1 Introduction

Object recognition is a core functionality of the proposed assistive system and plays a vital role in enhancing environmental awareness for visually impaired users. The objective of this module is to enable real-time identification of surrounding objects and provide meaningful audio feedback to the user in a fully hands-free manner. By using a trained object detection model, the system allows users to understand their surroundings without relying on manual cues. (Tapu et al., 2020; Zhao et al., 2019)

This chapter presents a detailed discussion of the design and preparation phase of the Object Recognition module. The focus is on system architecture, user interaction, dataset selection, preprocessing, annotation preparation, and final dataset structuring required for YOLO-based object detection.

3.2 System Overview and Functional Architecture

The Object Recognition module functions as an end-to-end pipeline initiated through voice commands. The system is designed with a modular and scalable architecture, enabling seamless integration with other assistive features such as OCR, face recognition, and navigation. Real-time performance is a key requirement, ensuring users receive immediate and reliable environmental feedback.

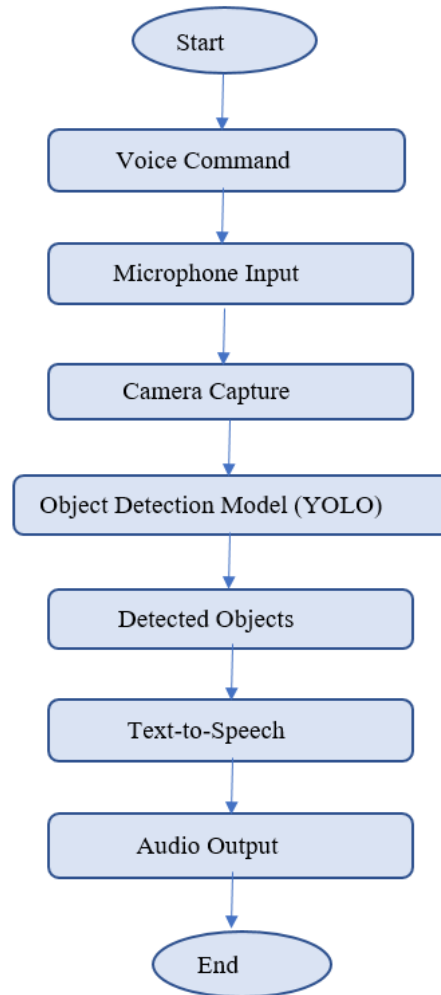


Figure 6 : System Overview and Functional Architecture

3.3 Limitations of Pretrained Models and Motivation for a Custom Dataset

During the initial development phase, a standard pretrained YOLOv8 model was evaluated for object detection tasks. Although the model demonstrated strong general-purpose detection capabilities, it did not adequately address the specific requirements of visually impaired users.

The pretrained model was originally trained on large-scale datasets such as COCO, which contain numerous object categories that are not directly relevant to assistive applications. Several limitations were identified during evaluation:

- Reduced detection accuracy for objects commonly encountered in the daily lives of visually impaired individuals.
- Presence of many irrelevant classes that increased computational overhead and reduced overall efficiency.
- Lack of optimization for the environmental contexts targeted by the proposed system.

These challenges highlighted the need for a custom dataset specifically tailored to objects that are meaningful for navigation, safety, and day-to-day interaction. Consequently, a custom training approach was adopted to improve both accuracy and practical usability.

3.4 Custom Dataset Development Using Roboflow

The most important task that should be considered while developing a good object recognition system is selecting and preparing a suitable data set. In our case, we have used a massive data set for object detection as a foundation for our research. However, from this data set, only those classes have been chosen that can be commonly found in the everyday life of visually impaired people. These classes could be, for example, chairs, doors, cars, bottles, etc.(Tapu et al., 2020; Zhao et al., 2019)

In order to mitigate the weaknesses in the generic pretrained network, it became necessary to build our own dataset using Roboflow, a platform offering functionalities for managing datasets, annotation, pre-processing and augmenting images.

The classes of objects included were chosen in light of how frequently these appear in the daily lives of visually impaired people. These include common indoor and outdoor objects such as doors, chairs, tables, bottles, cars, stairs, humans, among other things.

This dataset comprised of 77 different classes of objects. This ensured that only those categories of objects that matter to visually impaired people would be identified by the model, leading to greater efficiency and accuracy.

3.5 Data Augmentation

The entire process of dataset preparation has been done via a systematic data pipeline created in Google Colab Notebook. In the first stage, images and annotations for the particular classes have been downloaded. This ensures that there is no wastage of resources due to the non-availability of data.

In order to train the network to be effective in various environments, data augmentation methods were used via the augmentation pipeline of Roboflow. Data augmentation helps increase the variability in the data set by modifying existing images in the database.

Data augmentation methodologies used include:

- Horizontal image flipping to create alternate points of view.
- Brightness and contrast alteration to adapt to changes in lighting.
- Image rotation to compensate for changes in camera angle.
- Image cropping and zooming to create alternative points of view.

These augmentation strategies enhanced the model's resilience and improved its performance in real-world scenarios where environmental conditions can vary significantly.

3.6 Annotation and Preprocessing

Every image in the dataset was annotated with bounding boxes according to the identified object classes. The annotation process involved generation of annotations in YOLO format in which each label file contained the class ID and the normalized coordinates of bounding boxes.

The preprocessing steps that were taken to guarantee the quality of the data set include:

- Resize all images into one common size that is supported by YOLOv8.
- Validation of annotations to maintain data integrity.
- Removal of images with missing or damaged annotations.
- Classification checking to confirm class ID.

3.7 Annotation Conversion to YOLO Format

The original annotations were provided in XML-based format. Since the object detection framework intended for this project requires YOLO-compatible annotations, all labels were converted into YOLO format. (Redmon et al., 2016; Zhao et al., 2019)

The above conversion was done as follows:

Creating class names for each object

1. Indexing each class name with numerical index
2. Normalization of bounding boxes in relation to image size
3. Generation of .txt annotation files for each image

The generated .txt files contained class indexes as well as normalizations of bounding boxes, following the YOLO format.

3.8 Model Fine-Tuning

After annotation, the YOLOv8 model was fine-tuned on a custom 77-class dataset using transfer learning. Instead of training from scratch, pre-trained YOLOv8 weights were used as a starting point, allowing the model to retain learned visual features while adapting to the new task. Hyperparameters such as learning rate and batch size were adjusted across multiple training iterations.

This fine-tuning process improved the model's ability to detect and classify objects relevant to assistive applications, resulting in higher accuracy and better contextual performance compared to the original model.

3.8.1 Results

The proposed YOLOv8-based object detection model's performance was assessed over 77 object categories following 80 epochs of training. Table 1 presents the quantitative outcomes obtained on the validation set.

Table 1 Quantitative evaluation metrics of the proposed model on the 77-class validation dataset

Metric	Score	Interpretation
Precision	63.9%	Good — low false positives
Recall	48.3%	Fair — some missed detections
F1-Score	55.0%	Fair — balanced precision-recall
mAP@50	52.5%	Fair — moderate localization accuracy
mAP@50:95	37.7%	Weak __stricter IoU threshold challenge

As presented in Table 1, The precision rate of the model was 63.9%, reflecting its proficiency in reducing false positives. Nonetheless, the low values of 48.3% recall rate and 37.3% mAP@50:95 suggest that there is a need to improve the object detection capabilities especially with high localization thresholds. Figure 1 shows how the loss and mAP curves evolve for both training and validation during 80 iterations. It can be observed that although training losses tend to stabilize quickly, validation losses tend to increase steadily.

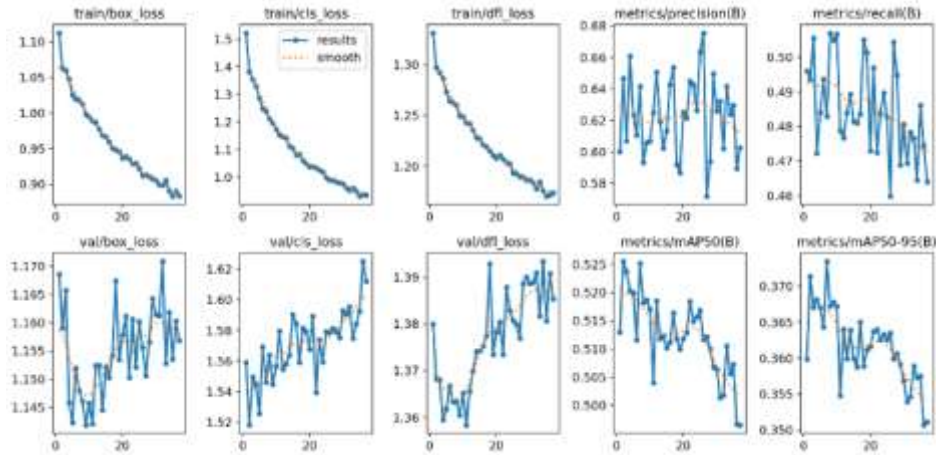


Figure 7 : Comparison of training and validation loss curves.

3.9 Limitations

While utmost care has been taken to prepare the dataset, the present implementation still has some limitations. The object recognition dataset includes only 77 classes of preselected objects and therefore does not include all real-life objects relevant to visually-impaired individuals. In consequence, any objects other than those included in the dataset may not be recognized while executing the program.

Moreover, the range of variations in the dataset is limited based on the amount and conditions under which the images are taken. Furthermore, limitations on computational power due to use of embedded devices may pose some challenges with respect to real-time object recognition for several objects at once.

Nonetheless, such limitations should not be an issue anymore in future stages of implementation.

3.10 Future Work

After completing dataset preparation and preprocessing, the next step is to develop a training pipeline for a real-time YOLO-based object recognition model. The model will learn to detect and classify objects relevant to visually impaired users.

Once trained, it will be integrated into the recognition pipeline for real-time video analysis and audio-based feedback. Future improvements include adaptive learning for expanding unseen object categories and enhancing accuracy over time.

Additionally, the module will be integrated into a mobile interface with voice command support, providing audio output and seamless interaction with other assistive features of the application.

3.11 Conclusion

This chapter provided an overview of the Object Recognition module, focusing on system design and dataset development. It covered the module structure, voice-based user interface, dataset creation and annotation, class filtering, data processing, and preparation of the YOLO dataset.

With the completion of the data set development process, there has been laid a solid foundation that is crucial for the training and implementation of the object recognition model. In designing the module, there has been emphasis on scalability, flexibility, and accessibility to enable the system adapt with time via features such as adaptive learning and mobile app development..(Tapu et al., 2020)

Chapter 4: Currency Detection Module

4.1 Introduction & Motivation

Money plays a vital role in daily life, where financial independence depends on tasks such as payments, budgeting, and purchases. For visually impaired individuals, while recognizing a banknote's denomination may be possible, distinguishing between different notes can be difficult and stressful without assistance. This can lead to uncertainty during transactions and potential financial or social discomfort.

The Currency Detection module in See Beyond Vision addresses this challenge by enabling automatic identification of Pakistani currency using a camera-based system. It recognizes all denominations regardless of orientation, lighting, or positioning using computer vision and deep learning techniques. This module enhances user independence and forms a crucial part of the overall assistive system.

4.2 Early Approaches and Model Evolution

The currency detection module was developed through multiple experimental stages, where early approaches highlighted limitations that guided improvements in dataset quality, model selection, and training strategy. Prior research shows that traditional image processing methods struggle with variations in lighting, orientation, and background, while machine learning approaches offer improved accuracy and reliability for camera-based currency recognition (Cs et al., 2022).

Deep learning advancements have further enhanced performance, with CNN-based detectors like YOLOv3 achieving strong accuracy and real-time detection under challenging conditions such as occlusion and illumination changes. YOLO-based models are particularly suitable for assistive applications as they enable fast, simultaneous detection of multiple objects (Joshi et al., 2020).

These insights informed the design of our system, which targets real-world usability, robustness, and accurate localization of Pakistani currency notes.

4.2.1 First Approach: Pre-Collected Dataset with YOLOv8n

The currency detection module initially used a dataset from Roboflow containing around 4,000 annotated images of Pakistani banknotes across eight denominations (Rs. 10, 20, 50, 75, 100, 500, 1000, and 5000). The dataset was chosen to introduce diversity in scale, background, and orientation, although variations in image quality, lighting, and annotation consistency were present due to multiple data sources. The model was built using YOLOv8n, a lightweight object detector suitable for real-time mobile applications, and trained for 150 epochs to learn from the dataset effectively.

Results and Observations

After training, the model produced the following performance metrics:

Table 2 Performance Metrics of the YOLOv8n Model Trained on the Pre-Collected Pakistani Currency Dataset

Precision	0.82
Recall	0.75
mAP@50	0.72
mAP@50–95	0.37
Overall Detection Accuracy	72%

The model achieved high precision, indicating correct classification of most detected notes, but recall was lower as several currency notes were missed. This issue was especially prominent for lower denominations (Rs. 10 and Rs. 20), which share similar colors and visual patterns, leading to misclassification or missed detections. The presence of duplicate, noisy, and low-quality images further degraded learning and reduced generalization. Additionally, overfitting was observed after around 150 epochs, where training accuracy continued to rise while validation performance plateaued or declined, indicating memorization rather than true feature learning.

Key Issues Identified

The major limitations observed during this approach include:

- **Dataset Imbalance and Inconsistency:** Some denominations of currency were either under represented or poorly labelled and biased the learning.
- **Generalization:** Visually similar notes were not always distinguished, particularly lower denominations.
- **High Computational Overhead:** Training on a large dataset increased training time without proportional gains in accuracy.
- **Overfitting:** Prolonged training caused the model to over-specialize on training samples, reducing real-world performance.

The first technique proved that although having a large amount of data may seem like an asset, the significance of good data lies far above this when it comes to reliable and accurate currency identification.

4.3 Current Solution: Custom Dataset with YOLOv8s

The third approach delivered the best performance for the currency detection module. It showed that neither large unstructured datasets nor very small structured ones are sufficient on their own; instead, a balanced dataset with real-world diversity is required. A custom dataset was built and augmented to closely reflect real-life conditions, which significantly improved the system's accuracy, stability, and reliability.

4.3.1 Custom Dataset Creation and Preprocessing

Unlike using pre-built datasets, an image collection was made through manually acquiring images of Pakistani currency notes, which reflect real-life situations in the best possible manner. It is essential that the model learns from situations similar to the ones visually challenged people face in their everyday life.

For this purpose, the image collection was made by taking into account:

- Different lighting conditions (bright, indoor, dark)
- Varying camera angles and directions
- Partial occlusion, for instance, partially folded or covered currency notes
- Background differences, such as table surfaces, fabrics, and wallets

Data Preprocessing and Augmentation

For improving the robustness and generalization of the training set, the dataset was first loaded into Roboflow for extensive processing and augmentation of the images. By doing so, the model is subjected to variations which occur in the real world.

The preprocessing methods used include:

- Automatic orientation
- Rotation
- Addition of blur and noise
- Adjusting brightness and exposure
- Varying hue and saturation

After undergoing all the preprocessing and augmentation, the initial 400-images dataset grew by more than three times, ending up at a size of roughly 962 images.



Figure 8 : Data augmentation examples showing rotation, brightness variation, blur, and noise applied to currency notes.

Dataset Split

The augmented data set was finally partitioned as follows to have three different sets for training, validation, and testing purposes:

- Training Set: 70%
- Validation Set: 20%
- Testing Set: 10%

Model Selection: YOLOv8s

In the case of the final deployment, YOLOv8s was chosen as the object detection model. YOLOv8s is an object detector created by Ultralytics that enables both object localization and classification in one pass.

As opposed to other models, YOLOv8s offers:

- More robust features because of its deep structure
- Better detection of fine details and textures
- Adequate inference time that can be applied in real-time applications
- Greater generalization to new data

All these benefits of YOLOv8s justify using it to detect Pakistani banknotes due to their visual similarity.

4.3.2 Training Results and Performance Evaluation

The YOLOv8s model was trained on the custom augmented dataset for 65 epochs, after which optimum results were obtained. The training process was carefully observed to ensure no degradation occurred during convergence. Below are the performance results from the model:

Table 3 Performance Metrics of the Final YOLOv8s Model Trained on the Custom Augmented Pakistani Currency Dataset

Precision	0.94
Recall	0.793
mAP@50	0.96
mAP@50–95	0.84
F1-score	86.0%

Total Samples: 82 , Correct Predictions (TP): 80 ,Wrong Predictions (FP + FN): 2

Final Accuracy = 97.56%

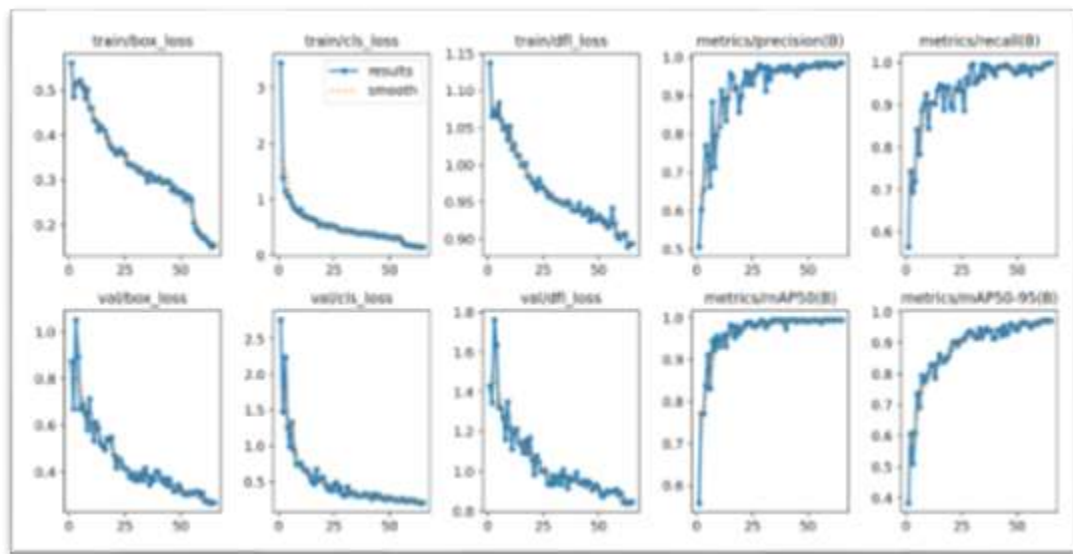


Figure 9 : Training and validation loss curves showing stable convergence and absence of overfitting and Mean Average Precision (mAP@50 and mAP@50–95) curves across training epochs.

Training Stability Analysis

Unlike other approaches, the loss functions for training and validation were consistent throughout the entire process of training. This indicates that there are no problems in learning and that the model is generalized well.

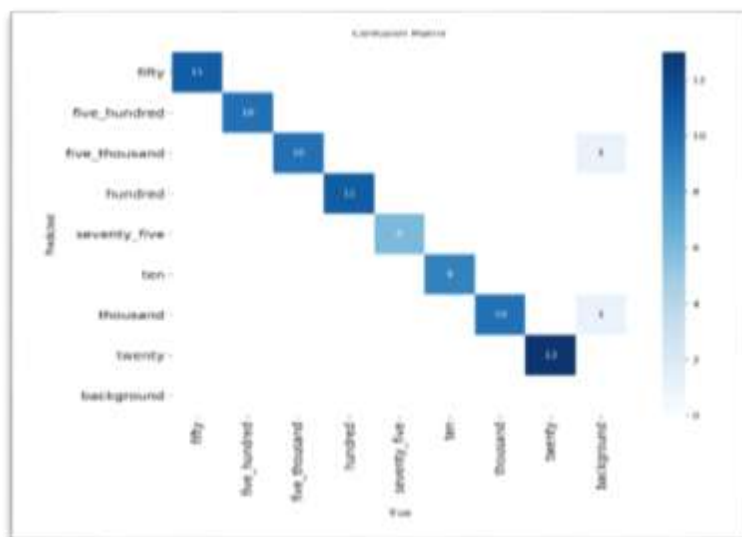


Figure 10 : Confusion matrix illustrating classification performance across all currency denominations.

Detection Output Results

To visually assess the model's effectiveness, various test images were processed by the trained model. The system accurately identified and categorized several currency values with great certainty



Figure 11 : Final detection results showing bounding boxes, class labels, and confidence scores for different currency notes.

4.4 Over-fitting and Under-fitting Analysis

When a model learns its training data too accurately, even including the noise in them, overfitting is said to happen. Underfitting happens when the model is not able to learn the underlying structure due to low model complexity or training. Earlier, overfitting was seen because there were not enough diverse datasets used, along with too many epochs used. Meanwhile, underfitting was experienced since the features were not learned effectively.

However, in the final approach, both problems did not exist because of the following reasons:

- Diverse and representative custom dataset used
- Effective data preprocessing and augmentation done
- Optimal number of epochs reached
- Model capacity given by YOLOv8s

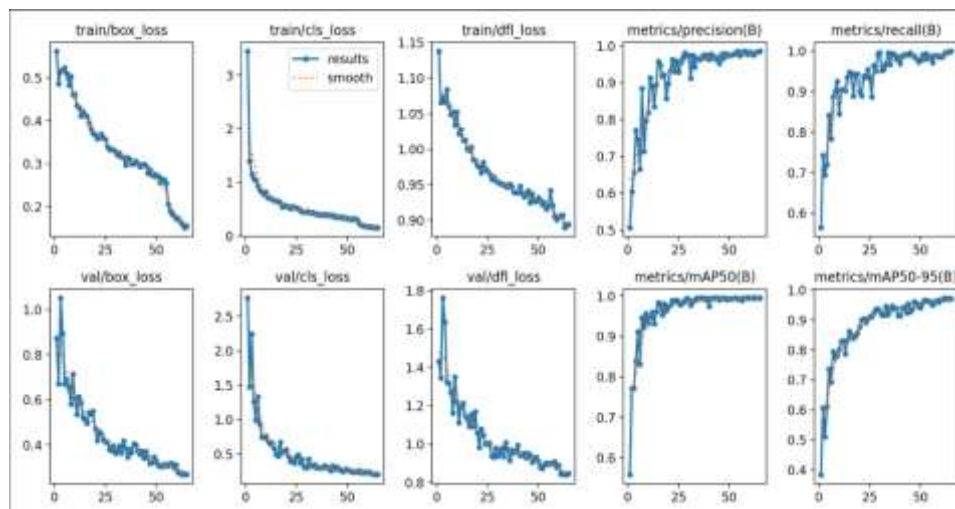


Figure 12 : Comparison of training and validation loss curves demonstrating balanced learning behavior.

4.5 Conclusion

The Currency Detection module of See Beyond Vision highlights the importance of a high-quality dataset over a large one. Earlier challenges such as misclassification and overfitting helped refine the system into a more reliable model. Using a curated dataset with YOLOv8s, the module achieves stable and accurate detection of Pakistani currency with strong generalization. It enables visually impaired users to independently identify banknotes, reducing financial risk and improving usability in real-world scenarios.

Chapter 5: Optical Character Recognition

5.1 Introduction

Optical Character Recognition (OCR) is a key component of the assistive system, enabling visually impaired users to read environmental text. Unlike document OCR, scene text recognition is more challenging due to varying lighting, complex backgrounds, motion blur, and arbitrary text orientations from head-mounted cameras. It is particularly useful for accessing real-world information such as signs, labels, and product text, helping users navigate indoor environments like grocery stores.

Scene-text recognition has therefore shifted toward deep learning-based end-to-end architectures that jointly perform text detection and recognition, significantly improving robustness under unconstrained real-world conditions as demonstrated in literature(Kai Wang et al., 2011)

This chapter presents the OCR approaches explored during the project, focusing on on-device and edge-based solutions suitable for real-time processing on a smartphone. The investigated methods include mobile-optimized machine learning frameworks and deep learning-based text detection models. Their performance, suitability, and limitations are discussed in detail.

5.2 OCR System Requirements

The OCR module was designed under the following constraints and requirements:

- Real-time operation using a camera mounted on smart glasses
- On-device processing using a smartphone, no dependency on cloud
- Robustness to scene text including signboards, labels, packaging, posters.
- Low latency and power efficiency

These constraints strongly influenced the selection of OCR approaches evaluated in this project.

5.2.1 Image Pre-Processing and Normalization Framework

A consistent preprocessing pipeline ensures optimal input quality for text detection and recognition from smart glasses. Raw frames are converted to grayscale to reduce noise and computation, followed by Adaptive Histogram Equalization to enhance local contrast and improve text visibility. The image is then padded and resized to a fixed resolution while preserving aspect ratio, ensuring compatibility with model input requirements and maintaining structural consistency across varying camera conditions.

5.3 Mobile ML-Based OCR using Google ML Kit

To enable real-time and on-device text recognition, the OCR module was first implemented using Google ML Kit Text Recognition API. ML Kit provides pre-trained models optimized for mobile devices and supports offline inference. It supports on-device execution with low latency, minimal setup and integration effort, good performance on printed, high-resolution text, suitable for battery-powered mobile platforms

5.3.1 Limitations Observed

Despite its practical advantages, ML Kit demonstrated several shortcomings when applied to scene-text:

- Reduced accuracy for distant or small text
- Weak detection under motion blur
- Limited robustness to non-uniform lighting
- Inconsistent results for complex backgrounds and stylized fonts

As a result, ML Kit was found to be reliable for **document-like or close-range text**, but insufficient for general-purpose scene-text recognition in assistive environments.

5.4 Deep Learning–Based Text Detection using YOLOv8

To improve robustness in complex scenes, a deep learning–based text detection approach was explored using the YOLOv8 architecture. The model was evaluated primarily for text region detection rather than direct recognition.

prior research in scene-text detection has demonstrated that single-stage detectors can effectively localize arbitrarily oriented text in natural scenes. The EAST

(Efficient and Accurate Scene Text Detector) framework directly predicts text geometries from full images using a fully convolutional network, eliminating complex intermediate steps and achieving high accuracy and efficiency(Zhou et al., 2017)

5.4.1 Approach

A lightweight YOLOv8 model was tested to identify text bounding boxes in real-time camera frames. Detected regions were intended to be passed to a separate recognition stage.

5.4.2 Observations

It was observed that there was strong localization performance for scene text, robust to scale variation and background clutter and effective for multi-text detection in a single frame. While YOLOv8 demonstrated promising detection capabilities, its deployment on mobile devices without model compression or optimization posed practical challenges.

5.5 Final OCR Approach: PaddleOCR

Based on the limitations observed in Google ML Kit, and generic object detection–based pipelines, the final OCR solution adopted for this project was **PaddleOCR**. PaddleOCR is an open-source deep learning-based OCR framework that provides an end-to-end pipeline for scene-text detection and recognition.

The primary motivation for selecting PaddleOCR was its strong performance on natural scene text, including text captured under challenging conditions such as perspective distortion, cluttered backgrounds, and varying illumination, conditions that closely resemble data captured from a camera.

PaddleOCR follows the end-to-end scene-text recognition paradigm, where text localization and transcription are optimized jointly, a strategy shown to outperform multi-stage pipelines in complex environments(Kai Wang et al., 2011)

5.5.1 PaddleOCR Architecture Overview

PaddleOCR employs a modular pipeline consisting of:

1. **Text Detection:** Localization of text regions using deep convolutional neural networks (e.g., DB-based detectors)
2. **Text Recognition:** Sequence-based recognition using CRNN-style architectures with CTC decoding.
3. **Post-processing:** Filtering, grouping, and normalization of detected text

This unified architecture allows PaddleOCR to outperform traditional OCR engines that rely heavily on handcrafted preprocessing.

5.5.2 Reasons for Selecting PaddleOCR

The following factors justified PaddleOCR as the final OCR solution:

- High accuracy on scene-text benchmarks
- Robust detection of multi-oriented and curved text
- Strong performance on low-resolution and partially blurred images
- Availability of pre-trained models
- End-to-end OCR without requiring external detection models

Unlike ML Kit, PaddleOCR demonstrated consistent recognition quality for distant signboards, packaging labels, and environmental text captured in unconstrained settings.

5.5.3 Experimental Observations

During experimental evaluation, PaddleOCR showed:

- Accurate text detection even in complex backgrounds
- Improved recognition accuracy for small and rotated text
- Better tolerance to illumination changes compared to ML Kit
- Reliable performance across multiple text instances in a single frame

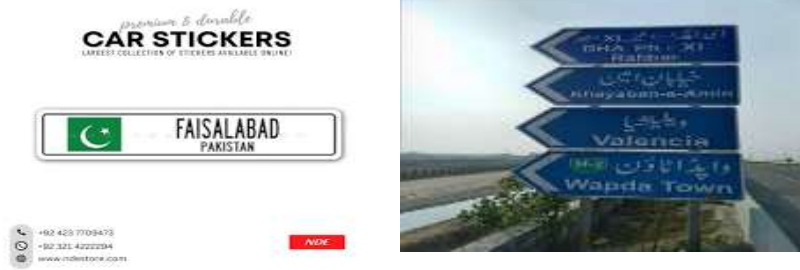


Figure 13 : Sample scene-text image captured in a real-world environment and processed using the PaddleOCR pipeline.

Figure 5.1 illustrates a qualitative example of scene-text recognition using the proposed PaddleOCR-based pipeline. The system successfully detected and recognized multiple text instances from a real-world image containing branding text, geographic information, and numerical data. High confidence scores were observed for most detected text regions, demonstrating robustness to varying font styles and background elements. Minor recognition errors were observed for stylized or partially occluded text, highlighting the challenges of unconstrained scene-text recognition. These results confirmed PaddleOCR’s suitability for real-world assistive OCR applications.

Table 4 Sample Text Recognition Results and Corresponding Confidence Scores Obtained Using PaddleOCR

Detected Text	Confidence Score
CAR STICKERS	0.96
FAISALABAD	1.00
PAKISTAN	1.00
92 321 4222294	0.89
NDE	1.00
Detected Text	Confidence Score
DHA Ph-XI	0.84
Rahbar	0.99
Khayaban-e-Amin	0.99
Valencia	0.99
Wapda Town	0.98

5.5.4 Limitations of PaddleOCR

Despite its superior accuracy, PaddleOCR introduced several practical limitations within the context of this project:

- **High computational cost**, making real-time execution on mobile devices challenging
- **Larger model size** compared to mobile-optimized OCR solutions
- **Inference latency** unsuitable for continuous video streams without optimization
- Requires **model conversion and acceleration** (e.g., TensorRT, ONNX, or Lite variants) for deployment on smartphones
- Increased **power consumption**, impacting battery life

Due to these constraints, PaddleOCR was primarily evaluated in an experimental and validation setting rather than fully deployed in real-time on-device inference.

5.5.5 Trade-off Analysis and Final Decision

The selection of PaddleOCR reflects a deliberate trade-off between **accuracy and deployability**. While ML Kit offered faster inference and easier integration, its accuracy on scene-text was insufficient for assistive use. PaddleOCR, on the other hand, delivered significantly higher recognition accuracy at the cost of computational efficiency.

Given the academic scope and constraints of the project, PaddleOCR was chosen as the **final OCR approach** to demonstrate the feasibility and performance ceiling of deep learning-based scene-text recognition.

5.6 Summary

This chapter explored multiple OCR approaches, progressing from traditional OCR techniques to mobile ML frameworks and deep learning-based methods. ML Kit was found to be limited in handling real-world scene-text. PaddleOCR emerged as the most accurate solution for unconstrained environments, despite its higher computational requirements. The findings from this evaluation provide a strong foundation for future work focused on model optimization and mobile deployments.

Chapter: 6 People Recognition

6.1 Introduction

People recognition is a key enabling feature for assistive systems designed for visually impaired individuals. The ability to identify known people in the surrounding environment can significantly enhance social interaction, confidence, and situational awareness.

The primary objective of this module is to recognize multiple known individuals in real time without relying on internet connectivity or cloud-based services. To ensure privacy and scalability, the system avoids storing raw facial images and instead relies on compact numerical representations of faces. This chapter presents the conceptual background and implementation details of the people recognition module.

6.2 Overview

The people recognition module is based on an embedding-based face recognition pipeline. Rather than training a classifier for a fixed set of identities, the system converts each detected face into a low-dimensional feature vector and performs similarity matching against a local database.

The main technologies used are:

- **OpenCV Deep Neural Network (DNN) Module:** Used for face detection and deep model inference, enabling deployment without specialized build tools.
- **FaceNet-based Embedding Model (OpenFace variant):** Generates 128-dimensional facial embeddings that capture discriminative facial features.
- **Cosine Similarity Measure:** Used to compare facial embeddings efficiently.
- **Local Embedding Database:** A lightweight JSON-based database that stores facial embeddings along with associated names.

This modular architecture allows the recognition model to be upgraded in the future without altering the overall system workflow.

6.3 Working of the People Recognition Module

The operation of the people recognition module proceeds through the following steps:

1. **Image Acquisition:** A live video stream is captured using webcam of PC.
2. **Face Detection:** Each frame is processed by a deep learning-based face detector implemented using OpenCV's DNN module. The detector identifies the facial region within the frame.
3. **Face Preprocessing:** The detected face is cropped and resized to a fixed resolution. Pixel values are normalized to match the input requirements of the embedding network.
4. **Feature Extraction:** The preprocessed face image is passed through the FaceNet-based neural network, producing a 128-dimensional embedding vector that uniquely represents the face.
5. **Similarity Matching:** The extracted embedding is compared with stored embeddings in the local database using cosine similarity.
6. **Identity Decision:** The identity corresponding to the highest similarity score is selected. If the score exceeds a predefined threshold, the person is recognized; otherwise, the face is classified as unknown.
7. **Output Display:** The recognized name is displayed on the video stream as the system output.

This workflow enables real-time recognition while maintaining computational efficiency and privacy.

6.4 Previous Implementation

An alternative approach used a Siamese neural network trained on the Labeled Faces in the Wild (LFW) dataset. A custom preprocessing pipeline organized the data into positive and negative image pairs, as well as anchor-positive-negative triplets for metric learning. This setup enabled the model to learn facial similarity instead of direct identity classification. For inference, a separate reference set was used, and the Siamese network performed face verification by comparing input images against stored samples to determine identity matches.

In this earlier approach:

- Faces were detected using the MTCNN algorithm.
- A Siamese network was trained to learn a similarity function between pairs of facial images.
- The model performed face verification by determining whether two input images belonged to the same person.

Although this approach showed reasonable performance for one-to-one face verification, it was not well-suited for the intended application. The main limitations included:

- Poor scalability for multi-person identification, as each query required multiple pairwise comparisons.
- Increased storage requirements due to the need to store several reference images per person.
- Reduced efficiency and reliability in real-time scenarios involving many known individuals.

Due to these constraints, the Siamese-based implementation was replaced with an embedding-based identification approach.

6.5 Challenges

Despite its effectiveness, the initial design and development of the people recognition module revealed several practical challenges:

- **Storage Overhead:** Early assistive face recognition systems often store multiple raw images per individual, which becomes storage-intensive as the number of known people increases. This approach is not scalable for real-world deployment.
- **Generalization Issues:** Verification-based Siamese models showed reduced robustness when tested on images captured under different lighting conditions, poses, and camera qualities compared to training data.
- **Computational Constraints:** Some deep learning frameworks and libraries required complex build tools on Windows systems, making rapid prototyping difficult.

- **Real-Time Performance:** Pairwise image comparison in Siamese networks increased latency as the database size grew.

These challenges motivated the transition to a lightweight, embedding-based recognition pipeline implemented entirely using OpenCV DNN modules.

6.6 Advantages of the Current Approach

The current embedding-based face recognition method provides several advantages over the previous implementation:

- **Scalability:** New individuals can be added to the system by storing their embeddings without retraining the model.
- **Storage Efficiency:** Only numerical embeddings are stored instead of raw facial images.
- **Privacy Preservation:** The absence of stored facial images reduces privacy concerns.
- **Real-Time Performance:** Cosine similarity computation is lightweight, enabling fast identification.

These advantages make the proposed approach suitable for assistive applications and future embedded deployment.

6.7 Transition to High-Performance FaceNet Architecture

Although a simple prototype route was provided by the original pipeline based on the standard OpenCV DNN module and the legacy OpenFace variant, real-world testing identified significant performance limitations. Due to its poor tolerance to scale changes, inability to recognize faces at sharp angles, and high false-positive rates in unrestricted illumination, the traditional DNN model has severe difficulties with real-world scene variations.

The basic feature extraction engine was modified to a specialized (Rutuja Bankar et al., 2022) in order to get beyond these restrictions and ensure true thesis-grade reliability for an assistive system. Facial similarity metrics are closely correlated with distances in a compact Euclidean space (geometrical points) that FaceNet translates facial features to. By utilizing strong deep learning representations, this method gets over the drawbacks of conventional, erratic face recognition systems.

6.8 Methodology

Using a localized, low-latency deep learning pipeline designed specifically for edge execution, the persons recognition module moves from raw image acquisition to deep-feature categorization. To separate the subject's face from complicated backdrops, a Multi-task Cascaded Convolutional Network (MTCNN) is fed raw frames from a live video stream. The Proposal Network (P-Net) for candidate selection, the Refine Network (R-Net) for false-positive filtering, and the Output Network (O-Net) for bounding box regression are the three cascading layers that the MTCNN passes the frame through. The five main facial landmarks—eyes, nose, and mouth corners—are mapped at the end of this step. A 2D affine transformation is then used to horizontally align, scale, and crop the facial patch in order to remove head tilts.

The FaceNet deep convolutional neural network engine then processes the aligned facial patch. FaceNet creates a highly discriminative 128-dimensional embedding vector by directly mapping high-level structural information into a low-dimensional Euclidean space, eschewing traditional classification layers. A triplet loss function is used to optimize this embedding space, maximizing the distance between distinct persons (anchor and negative pairs) and minimizing the distance between matching identities (anchor and positive pairs). Robust one-shot learning is made possible by this mathematical arrangement, which enables the system to enroll and identify new users with small image samples without having to retrain the underlying network. In order to create clear dividing hyper-planes between different target profiles for the final identity decision, a Support Vector Machine (SVM) classifier with a linear kernel is layered on top of the FaceNet outputs. The freshly extracted 128-dimensional vector is compared to the training SVM parameters during real-time inference. The verified identification label is successfully returned and shown on the stream if the resulting classification

probability meets a rigorously predetermined verification threshold; if not, the face is safely categorized as unknown in order to maintain system security and reliability.

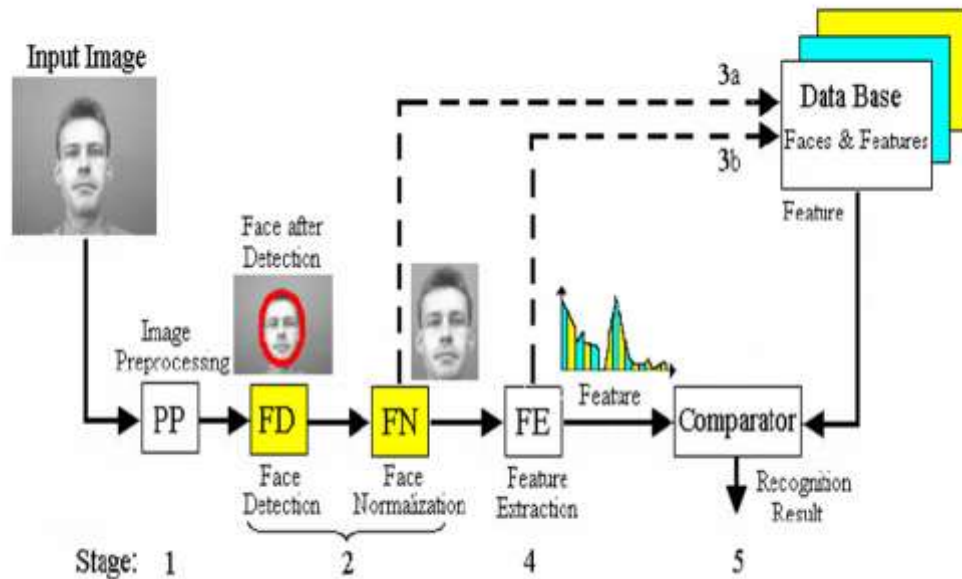


Figure 14 : Operational pipeline of the FaceNet face recognition system from preprocessing to verification.

6.9 Detailed FaceNet Workflow

A simplified deep learning pipeline designed for real-time edge execution powers the improved People Recognition Module. The following architectural steps are used by the system to process video frames:

- Facial Landmark Detection using MTCNN:** A Multi-task Cascaded Convolutional Network (MTCNN) processes the frame rather than using generic bounding boxes. Three separate neural networks make up the cascade structure used by the MTCNN algorithm: the Proposal Network (P-Net), which creates candidate areas; the Refine Network (R-Net), which eliminates false candidates; and the Output Network (O-Net). The locations of both eyes, the nose tip, and the mouth corners are the five crucial facial landmarks that are localized at the end of this cascade.

- **Geometric Alignment and Preprocessing:** A bounding box is created using the coordinate points of the identified landmarks in order to crop the face closely from the background, tightly cutting it on the face field without requiring intricate 3D corrections. The clipped face patch is stretched and scaled to fit the deep learning model's exact input dimensions.
- **Geometric Alignment and Preprocessing:** A bounding box is created using the coordinate points of the identified landmarks in order to crop the face closely from the background, tightly cutting it on the face field without requiring intricate 3D corrections. The clipped face patch is stretched and scaled to fit the deep learning model's exact input dimensions.
- **Deep Feature Extraction:** The deep Convolutional Neural Network processes the preprocessed face image. The network directly produces a succinct, highly discriminative 128-dimensional embedding vector after extracting abstract facial structures. The most important spatial features of the face in Euclidean space are captured by this vector.
- **Optimized Learning with Triplet Loss:** By using a triplet loss training technique, the underlying FaceNet model attains strong intra-class separation. This function maximizes the distance between an anchor picture and a negative image (different identity) while simultaneously minimizes the distance between an anchor image and a positive image (same identity). This makes sure that while diverse faces are pushed far apart, similar faces are grouped closely together.

6.10 Experimental Observations and Advantages of FaceNet

Benefits and Experimental Findings of FaceNet Significant performance improvements were obtained by switching to complete FaceNet architecture in real-world criteria:

- **Robustness to Illumination and Pose Shifts:** Despite severe shadows, backlighting, and non-frontal viewing angles, the model effectively recognizes faces because of the deep structural representations that the Inception-ResNet backbone learnt during its huge triplet-loss training phase.
- **Superior Inter-Class Separation:** The architecture optimizes embedding spaces so that various individuals' vectors are pushed widely apart while those of the same individual are clustered closer together. This significantly reduced the false-acceptance rate that the old OpenCV DNN module had.
- **Ultra-Lightweight Storage:** A text-based JSON registry only needs to store 128 decimal values in order to store a single identity. This makes it extremely scalable for wearable devices with limited resources by avoiding the need to archive large image files.

6.11 Conclusion

This chapter traced the design evolution of the People Recognition Module, highlighting the practical lessons that guided its development. Early exploration into Siamese verification networks highlighted major multi-person scaling issues, while the secondary OpenCV DNN pipeline proved vulnerable to real-world ambient conditions.

The integration of a FaceNet pipeline solved these challenges, delivering an optimized, highly accurate, and privacy-preserving face recognition engine. By isolating compact numerical embeddings and evaluating them via fast similarity metrics, the module achieves robust real-time identification performance. This establishes a firm technical foundation for translation into low-power mobile runtimes to support visually impaired individuals natively at the edge.

Chapter 7: Smart Blind Cane with Dual Ultrasonic Sensors and Haptic Feedback

7.1 Introduction

One of the most significant issues that visually impaired people deal with is mobility. Traditional white canes work well for identifying objects that are in close proximity to the cane, but they might not be able to warn of problems that are farther away. This project's Smart Blind Cane combines haptic feedback and ultrasonic detection to produce a useful and reasonably priced mobility aid. When obstacles are found within predetermined ranges, the system constantly scans the environment and vibrates to notify the user.

7.2 Problem Statement

Traditional mobility aids cannot reliably detect distant obstacles, leaving users with limited reaction time and increasing collision risk. This research proposes a low-cost, low-complexity smart cane that detects obstacles before contact and provides real-time feedback for proactive navigation. The system distinguishes between near and far objects while optimizing power consumption and maintaining a simple, portable design.

7.3 Objectives

The main goal of this project is to develop an intelligent, portable, and low-cost assistive device to enhance the safety and independence of visually impaired users. It uses ultrasonic sensing to detect obstacles in the user's path and provides real-time haptic feedback through a vibration motor when hazards are detected, enabling timely direction changes. Overall, the system improves situational awareness and supports safer, more confident navigation.

7.4 Hardware Components

The system consists of the following components:

1. Arduino Nano

The system's central processing unit is the Arduino Nano. It takes measurements of distance from the ultrasonic sensors, analyzes the information, and adjusts the vibration motor.

2. Ultrasonic Sensor 1 (Near-Range Sensor)

The first HC-SR04 ultrasonic sensor is set up to track obstructions up to 20 centimeters away. This sensor is in charge of identifying current threats that need to be addressed immediately.

3. Ultrasonic Sensor 2 (Far-Range Sensor)

Obstacles within around 60 cm are monitored by the second HC-SR04 ultrasonic sensor. Before the obstruction enters the close detecting zone, it issues an early warning.

4. Coin Vibration Motor

The haptic feedback mechanism is the vibration motor. Vibrations offer covert and efficient input without upsetting those nearby, as opposed to loud alarms.

5. Transistor Driver Circuit

An Arduino pin cannot safely supply the amount of electricity needed for the vibration motor. In order to drive the motor, an NPN transistor is utilized as a switching device.

6. Protection Diode

A flyback diode is connected across the motor terminals to protect the circuit from voltage spikes generated when the motor is switched off.

7.5 Methodology

To see if the proposed system is efficient we compared it to a study by Kim from (Kim, 2025). This study made a walking cane with many sensors. Both systems use the Arduino platform. Provide feedback through vibration but they are built and sense things in very different ways.

The system made by Kim uses one sensor to find things in front of it. It has a special belt with a sensor to track how the cane is moving. Our system is different because it does not need a belt. All the parts are in the cane itself. Of tracking how the cane is moving our system uses two sensors to find things that are close and far away. This lets us warn the user in ways using the sensors and some software.

The system made by Kim also uses amplifiers to control the vibration motors, which are on the handle and the belt. Our system is simpler. We use a kind of transistor to control the motors and we have a protection system to prevent damage from voltage spikes. This makes our system more reliable and easier to build. We think our smart walking cane system is an idea because it is simple and efficient. The proposed system and Kims system both have their ways of doing things but our system is better, in some ways. The proposed system is a walking cane system that uses the Arduino platform and provides feedback through vibration just like Kims system.

7.5.1 Obstacle Detection

The microcontroller causes the haptic system to produce sporadic vibration pulses when it detects an environmental obstruction between 20 cm and 40 cm. The control signal is cycled high and low at a set frequency to produce this pulsating effect. By alerting the user to an impending object, this operational state functions as a non-urgent early warning signal, allowing them to proactively plan their movement without having to stop right away.

7.5.2 No Obstacle Condition

The vibration motor stays completely disabled if the real-time sensor data show that the path is clear and there are no obstructions inside the near or far threshold zones (beyond 40 cm). By removing superfluous sensory feedback while navigating safe areas, this idle state preserves battery life and avoids user fatigue.

7.6 Circuit Design

All connected hardware components are designed with accurate signal routing, stability, and safety in mind. The following is a definition of the interface details:

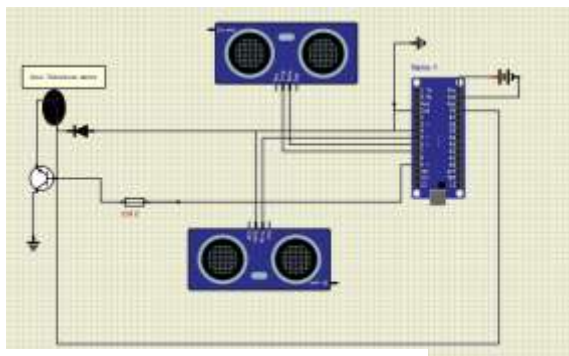


Figure 15 : Circuit Design of the Smart cane

7.6.1 Configuration of the Sensor: To avoid data collisions, the dual-sensor architecture makes use of specific digital channels on the Arduino Nano. Digital pins D6 (Trigger) and D7 (Echo) are directly connected to the near-range ultrasonic sensor. Digital pins D4 (Trigger) and D5 (Echo) are simultaneously routed to the far-range ultrasonic sensor. The microcontroller may pulse and poll each sensor in turn without signal cross-talk thanks to this independent pin assignment.

- **Driver Logic for Motors:** An external switching circuit is used because the vibration motor requires more current than an Arduino I/O pin can supply. An NPN transistor controlled by digital pin D9 acts as a switch, driving the motor. When a HIGH or PWM signal is applied, the transistor enters saturation, allowing current to flow from the power supply through the motor to ground.
- **Protection of Inductive Loads:** A flyback diode is connected in reverse bias across the vibration motor terminals to protect the circuit from voltage spikes. When the transistor switches off, the motor generates back-EMF due to its collapsing magnetic

field. The diode safely dissipates this current, preventing damage to the transistor and Arduino.

- **Typical Grounding:**The NPN transistor switching network, the motor, the microcontroller, and both HC-SR04 sensors share a single common ground connection in order to create a dependable electrical reference throughout the entire system. This guarantees reliable sensor logic and noise-free operation by removing ground loops and floating voltages.

7.7 Limitations

The system has several limitations due to ultrasonic sensing and portable hardware design. Sensor accuracy can decrease on soft or textured surfaces that absorb sound waves, and performance may also be affected by heavy rain or extreme weather conditions. It is limited to detecting obstacle distance without identifying object type or shape. In addition, battery life is constrained by overall usage and the frequency of vibration motor activation.

7.8 Future Improvements

Future versions of the smart cane could include GPS-based navigation with real-time voice guidance, along with lightweight on-device or camera-based ML for improved object detection. Hardware upgrades may feature an efficient rechargeable battery system, Bluetooth-enabled app controls, and a water-resistant enclosure to enhance durability and reliability in different weather conditions.

7.8 Conclusion

The Smart Blind Cane is an example of how sensor technology and embedded systems may increase the mobility of people with vision impairments. The device offers both early warnings and instant obstacle alarms by integrating two ultrasonic sensors with haptic feedback. The design can improve user safety while navigating and is inexpensive and simple to install. The study demonstrates how microcontrollers and sensor technologies may be used practically in accessibility and assistive healthcare solutions.

Chapter 8 App Development

8.1 Introduction

The backend for the proposed Assistive Vision System acts as the main computational powerhouse behind the application. Implemented in the form of a high-speed API built in Python through FastAPI, its role entails handling multi-modal image processing tasks, conducting deep learning inference, and serving structured JSON responses to the mobile Flutter client.

In order to address the challenging needs of a real-time assistive application, the backend is designed on the premises of modularity, scalability, speed, and clear separation of concerns. The different functionalities of Artificial Intelligence (AI) and Machine Learning (ML) are decoupled into service layers. This allows for smooth upgrades and extensions in the future without any chance of breaking other parts of the application.

8.2 Technology Stack Rationale

The choice of components for backend technology stack was defined by several factors – specifically, the need for asynchronous computation, fast prototyping, and optimal overhead computation:

FastAPI Framework: Used as the primary web framework due to its inbuilt asynchronous server gateway interface capability, efficient concurrency management ability, and automatic creation of interactive openapi documentation.

Python Language: Chosen as the main programming language owing to its highly evolved library ecosystem with deep learning and data manipulation abilities.

RetinaFace Algorithm: Used as the facial recognition algorithm for its superior single stage pixel level prediction and landmark extraction capabilities.

Facenet TensorFlow Lite model: Used for generating highly distinctive 512 dimensional identity embeddings, optimized by using quantization.

EasyOCR Text Recognition Engine: Used as a deep learning based engine for unconstrained text recognition from natural scenes.

YOLOv8: Chosen for two different use cases involving customized training, specifically real-time currency note detection, and blind assistance object detection.

OpenCV library: For all low level image manipulations such as color transformations and affine transforms on images.

8.3 Backend Architectural Design

The design utilizes a Layered Software Pattern to maintain loose coupling among the client communication interface, the internal business logic, and the primary machine learning models.

8.3.1 Architectural Pipeline Execution

The process moves in a step-by-step way through the system from one part to the next. It starts at the client side, where the Flutter app takes video frames from the camera and sends them through secure HTTP POST connections as multipart form data.

This payload is right away caught by the FastAPI API Routing Layer, which checks the data against a schema and cleans up the request. Once the verification is done, the controllers send the raw image data to the special Service Layer. In this case, the business logic decides which deep learning process should run depending on the incoming path.

At the end, the services talk directly to the Model Inference Layer, sending the preprocessed tensors into the memory blocks of RetinaFace, FaceNet, EasyOCR, or YOLOv8, and gathering their raw prediction results for the final processing.

8.4 Project Structure and Modularity

The workspace is set up in a way that uses a domain-driven micro-package system. This setup helps prevent any issues with circular dependencies and lets developers work on their parts independently. The main application folder is divided into separate folders for different functions:

- The API package includes specific routers for handling endpoints and has clear rules for validating data.
- The Core package includes tools for handling global settings and managing dependency injection frameworks.
- The Services package serves as the main group of functions, with separate sections for face detection, aligning faces, creating embeddings, matching with a database, recognizing text in scenes, classifying currencies, and tracking environmental hazards.
- The Assets package offers a localized file system repository for storing frozen serialized model weights and registered identity embedding files.

This modularity means that any service can be completely rewritten or swapped with a new version without affecting how the network handles things or changing the features next to it.

8.5 Centralized Configuration Management

To improve maintainability, a configuration system using **Pydantic Settings** centralizes file paths, environment variables, and model parameters. It loads model weights, validates settings at startup, and enforces confidence thresholds before runtime. This ensures type safety, prevents configuration errors, and simplifies deployment across development and production environments.

8.6 Service Dependency Injection Container

To avoid the problem of slow processing caused by reloading big deep learning model files each time a client sends a request, a custom application container was created to handle dependency injection.

8.6.1 Cold Boot State and Lifecycle Management

On startup, the backend performs a cold boot where configuration files are loaded and all model weights are initialized into memory. These models are stored as shared singleton instances within the application container. Incoming API requests reuse these in-memory model references, avoiding repeated disk I/O, improving latency, reducing memory overhead, and supporting cleaner unit testing through modular separation.

8.7 Multi-Stage Face Recognition Workspace

The face analysis system is broken down into a four-step computer vision process that is very organized. It's designed to ensure accurate identification even when there are different lighting conditions or angles in real-life situations.

8.7.1 Subsystem Pipeline Mechanics

Face Detection Phase: The raw incoming color image is sent into the RetinaFace network for face detection. The model identifies where a human face is and finds five exact points: the left eye, right eye, nose tip, left side of the mouth, and right side of the mouth.

Geometric Face Alignment Phase: The next step is geometric face alignment, where the identified key points are sent to a preprocessing system that figures out the angle between the two eyes. Using this method, an automatic affine transformation matrix is created with OpenCV to rotate and adjust the face image so it's aligned straight up, which corrects any side tilts of the head.

Embedding Vector Generation Phase: The face image is adjusted to a standard size so it can be used as input for the FaceNet network. The image goes through the deep convolutional layers of the quantized TensorFlow Lite model, which

produces a unique 512-dimensional vector made up of real numbers that represent the face's identity numerically.

Vector Metric Matching Phase: The new vector is checked against a local database of user embeddings that were already registered, using the Cosine Similarity method. The engine calculates the product of the vectors and then divides it by the product of their lengths. If the calculated distance meets the set match limit, a positive identity name is connected to the response packet.

8.8 Optical Character Recognition Module

The OCR module takes text that's scattered in busy pictures and turns it into clear, easy-to-read computer text, helping people understand documents, signs, or medicine instructions.

8.8.1 OCR Processing Stages

The process begins with image preprocessing, where the frame is converted to grayscale and enhanced using bilateral filtering to improve contrast. The image is then processed by EasyOCR, which detects text regions and extracts characters using a recognition network. Multi-line text is reorganized into a readable sequence and cleaned into a structured format for efficient conversion into speech by the text-to-speech system.

8.9 Custom Currency Detection System

The currency module was designed to enable visually impaired users to carry out financial transactions autonomously without depending on outside help.

8.9.1 Model Tuning and Currency Classification

A customized YOLOv8 model, trained on a regional banknote dataset, detects and classifies seven Pakistani Rupee denominations (10, 20, 50, 100, 500, 1000, and 5000 PKR). The model predicts note classes and bounding boxes while filtering low-confidence detections to ensure reliable currency recognition.

8.10 Blind Assistance and Threat Prioritization

The blind assistance feature acts as the primary environmental awareness system, identifying objects in the user's vicinity to recognize pathways, obstacles, and structural dangers.

8.10.1 Object Segmentation and Dynamic Priority Parsing

A customized YOLOv8 model detects everyday objects and structural elements, which are prioritized into three categories to reduce information overload. **High-priority** objects (e.g., vehicles, stairs, potholes) trigger immediate audio and haptic alerts. **Medium-priority** objects (e.g., pedestrians, doors, animals) are reported sequentially, while **low-priority** objects (e.g., chairs, tables, mugs, laptops) are announced only during user-initiated scene scans.

8.11 REST API Layer Implementation

FastAPI controllers manage image uploads, validate requests using Pydantic models, and invoke the appropriate computer vision services. The results are returned as a consistent JSON response containing predictions, text, and confidence scores, enabling the Flutter frontend to process responses and provide low-latency user feedback.

8.12 Error Resiliency Framework

The backend uses comprehensive exception handling to maintain service availability. It detects invalid files, unreadable images, missing faces, and model timeouts, converting errors into user-friendly JSON responses with tracking codes instead of interrupting the server. This allows the mobile client to handle failures without affecting the user experience.

8.13 Computational Performance Optimizations

To attain real-time processing speeds on standard hardware, multiple optimization strategies were incorporated into the backend architecture:

- **Model Compression through Quantization:** The large FaceNet model was converted into an 8-bit integer-quantized TensorFlow Lite format, decreasing storage requirements and speeding up CPU performance with minimal impact on accuracy.
- **Singleton Pattern Implementation:** Limiting model instantiation to the application startup phase reduces runtime resource usage and avoids delays in garbage collection.
- **Vectorized Distance Calculations:** Match-verification computations utilize parallelized NumPy array metrics, executing database queries in milliseconds.
- **Asynchronous Context Switching:** Endpoints use non-blocking input-output loops to manage ongoing client image uploads effectively without hindering the primary execution thread.

8.14 Security Implementations

The backend ensures secure and reliable operation by limiting upload sizes to prevent DoS attacks and processing images in memory instead of storing them on disk. In production, internal stack traces are hidden to prevent sensitive system information from being exposed during exceptions.

8.15 Summary

This chapter outlined the structural execution of the FastAPI-powered backend system created for the Assistive Vision System. Utilizing a multi-layered, decoupled service framework supported by localized dependency injection, the system combines intricate deep learning pipelines such as RetinaFace, FaceNet TFLite, EasyOCR, and optimized YOLOv8 networks into a unified structure. Backed by strong configuration management and error-handling frameworks, this backend provides a quick, reliable, and secure solution designed for the high-concurrency demands of real-time assistive mobile apps.

Chapter 9 Frontend Development

9.1 Introduction:

The system's frontend is a Flutter mobile application that offers a completely voice-activated, hands-free interface for users with visual impairments. The application constantly monitors for a wake word, processes verbal instructions, takes photos with the device camera, transmits those photos to the backend for AI analysis, and transforms the results back into vocal output. The application additionally offers a touch-based alternative for visually impaired assistants and separate displays for every assistive feature (text reading, object detection, currency recognition, face identification, and scene analysis).

The frontend comprises five essential subsystems that collaborate in an ongoing cycle:

- Voice Recognition (Speech-to-Text / STT) — activation phrase and command detection
- Intent Recognition — connects detected speech to a specific action
- Camera Utilization — one common camera employed throughout all modules, featuring a native fallback for locked-screen functionality.
- Backend Interaction — REST API requests to the FastAPI backend for every assistive capability
- Voice Output (Text-to-Speech / TTS) — transforms outputs back into audible sound.

9.2 Voice Input: Speech-to-Text (STT)

Voice input is managed through the `speech_to_text` Flutter package, which is encapsulated within a specialized `SpeechService` class. This service offers two listening modes utilized by the primary voice loop in `HomeScreen`

9.2.1 Wake Word Listening

`listenForWakeWord()` initiates an ongoing speech recognition process with partial outcomes activated. Each partial transcript is verified against the wake phrase “hey app”. This enables the assistant to reply immediately when the phrase is recognized, without needing to wait for the user to finish talking.

9.2.2 Command Listening

After detecting the wake word, the app gives a brief response (“Yes?”) and transitions to `listenForCommand()`, which awaits a single final transcript (`partialResults: false`). This acknowledged phrase — such as “read the text”, “detect currency”, or “who is this” — serves as the input for the Intent Detector.

9.2.3 The Voice Loop

The complete interaction is managed by an asynchronous loop (`_runLoop`) that rotates through four states, depicted by the `ListenState` enum:

1. Resting: awaiting the activation phrase "hey app"
2. waitingCommand: activated by wake word, asking the user for a command
3. Processing: a command has been received and is currently being executed (camera capture + API call)
4. Speaking: the outcome is being articulated through TTS.

Once processed, the loop automatically reverts to the idle state, ensuring the assistant is perpetually prepared for the next "hey app" without requiring any button presses. The speech recognizer is re-initialized every 10 cycles to avoid the Android STT engine from becoming unresponsive during extended sessions.

9.3 Background Operation:

By default, Android halts an app's access to the microphone and camera immediately when the screen is locked or the app is minimized. To ensure this assistant can be used hands-free (for instance, with the phone in a pocket), a native Android foreground service (`AssistantService`) is initiated upon app launch.

- In the Android manifest, the service is defined with `foregroundServiceType="camera|microphone,"` in addition to the permissions `FOREGROUND_SERVICE_CAMERA` and `FOREGROUND_SERVICE_MICROPHONE`.

- Launching the service triggers a continuous, low-priority alert (“Assistant is active”), indicating to Android that the application is engaged in a persistent task and must not be halted.
- A MethodChannel (com.example.frontend/service) connects Flutter with the native Android layer, enabling Dart code to invoke startService() when the app initializes.

This integration enables the speech recognizer to continue detecting "hey app" even if the screen is off, and permits the camera capture system to revert to a native solution when the Flutter camera preview isn't accessible.

9.4 Camera Capture Module

Each of the five support modules needs a picture of the user's environment. To prevent repeating camera logic, the app utilizes a single shared CameraProvider (a ChangeNotifier) that is instantiated once at the app's root and injected through the Provider package. Each screen accesses the same CameraController instance via context.watch(), guaranteeing that only a single camera session is active at any given time. This architectural design adheres to the fundamental concepts of isolating structural components and optimizing resources emphasized in modern Flutter development frameworks (Aung et al., 2024). The literature shows that creating a consistent hierarchical framework for Flutter components is crucial for ensuring application stability and avoiding resource excess (Aung et al., 2024). Utilizing this centralized widget method rather than creating separate camera controllers for various view screens allows the system to maintain a highly optimized, responsive, and lightweight hardware capture pipeline.

9.4.1 Foreground Capture

While the app is active, CameraProvider.takePhoto() reinstates the live preview, momentarily displays a white “capture flash” overlay for user feedback, and invokes CameraController.takePicture(), which returns the file path of the taken JPEG.

9.4.2 Background / Locked-Screen Capture

When the app runs in the background or with the screen locked, it uses a native Camera2 API implementation (CameraHelper.kt) instead of Flutter's CameraController to capture images. The captured photo is saved and its file path is returned to Flutter via the capturePhoto MethodChannel. HomeScreen selects the appropriate capture method based on the app's lifecycle state using WidgetsBindingObserver.

9.5 Intent Detection

The IntentDetector class transforms the free-form recognized speech into a specific VoiceIntent enum value through keyword matching. This maintains the rest of the application separate from the specific wording the user employs.

- “read”, “scan”, “text”, “book”, “ocr” → VoiceIntent.read (Optical Character Recognition)
- "item", "which is", "recognize", "find" → VoiceIntent.object
- “money”, “currency”, “note”, “cash”, “rupee” → VoiceIntent.currency
- “face”, “who is”, “identify”, “individual” → VoiceIntent.face / individuals
- “setting”, “depict”, “environment” → VoiceIntent.scene
- “assist”, “how can” → VoiceIntent.help (displays available commands)
- “halt”, “cease”, “abort” → VoiceIntent.stop
- anything else → VoiceIntent.unknown (the assistant requests the user to repeat the command)

After an intent is recognized, _handleCommand() either executes the relevant capture-and-analyze function directly or directs the user to the associated specific screen (for instance, the Face Recognition screen, which also includes the face enrollment/registration interface).

9.6 Backend Communication

9.6.1 Optical Character Recognition and Object Recognition:

The Read Text (OCR) module utilizes the POST `/api/ocr/read` endpoint to transmit a JPEG image, and the output "text" field is articulated word for word. Likewise, the Object Detection module transmits an image to POST `/api/object/detect` and utilizes the "objects" list to identify the top 3 labels, their location, and confidence score.

9.6.2 Monetary and Facial Modules:

The Currency Detection module sends a POST request to `/api/currency/analyze` with an image to announce the total sum and quantity. The Face Recognition module analyzes photos through POST `/api/face/analyze` to provide names or totals, whereas Face Registration utilizes POST `/api/face/register?name=` with a name parameter to convey a confirmation message.

9.6.3 Scene Depiction and Primary URL:

The Scene Description module transmits a photo to POST `/api/blind-assist/analyze` and verbalizes the scene overview. All of these requests utilize a configurable base URL (`ApiConfig.baseUrl`) that can be modified during the build process, enabling the same APK to link to various backend servers without the need for recompilation.

9.7 Text-to-Speech (TTS)

Speech feedback is generated using the `flutter_tts` package, initially set up at startup with a US-English voice, a somewhat lowered speech rate (0.45) for better clarity, maximum volume, and standard pitch. The `_speak()` helper encompasses each TTS call, allowing the rest of the app to easily await `_speak(text)`:

- First halts any active speech recognition to prevent the microphone from capturing the device's own audio output.
- Transitions the interface to the speaking mode (displayed to the user as an orange "Speaking..." signal accompanied by a volume icon).
- Invokes `_tts.speak(text)` and awaits either the completion callback from the platform or a safety timeout (relative to the text length), ensuring the

loop does not become unresponsive if the TTS engine does not trigger its callback.

- Introduces a brief pause afterward before continuing to listen, ensuring the microphone does not capture the final part of the speech.

This template is applied to all verbal replies: affirmations (“Yes?”), update messages (“Scanning, please hold”), error notifications (“Camera malfunction, please retry”), and — crucially — the ultimate outcomes delivered from the backend (the OCR output, the inventory of identified items, the overall currency sum, identified names, or the description of the scene).

9.8 End-to-End Pipeline

Voice Activation and Intent Recognition: The process begins when the user says the wake word, “**Hey App.**” The Speech Service continuously listens for this phrase and remains in the listening state until it is detected. Once recognized, the application responds with “**Yes?**” using Flutter TTS. It then captures the user’s voice command, converts it into text, and forwards it to the Intent Detector. The Intent Detector analyzes the text for keywords and maps the command to the appropriate **Voice Intent** enum.

Guidance and Picture Taking: After the intent is identified, `Navigator.push()` opens the corresponding screen (Read, Object, Currency, Face, or Scene). The system then captures an image using `CameraProvider.takePhoto()`, either through the shared camera controller or the native Camera2 API when running in the background or with the screen locked.

Server-Side Processing and Direction: The captured image is sent to the appropriate FastAPI endpoint via an HTTP multipart/form-data POST request. The backend processes the image using ML models (OCR, YOLO, or VLM) and returns the results in JSON format.

Processing Responses and Generating Output: The Dart service layer parses the JSON response, updates the UI with a semantic live region, and uses `FlutterTTS.speak()` to deliver the result as speech. After completion, the system resumes listening for “**Hey App,**” while the foreground service keeps the camera and microphone active even when the screen is locked.

9.9 Module Walkthroughs

9.9.1 Read Text (OCR)

User says “Hey App” → “interpret the text”. The application takes a picture, transmits it to `/api/ocr/read`, and gets the extracted text in return. When the text box contains information, it appears on the screen and is spoken aloud exactly as written. If no text is found, the assistant prompts the user to verify that the page is adequately illuminated and attempt again.

9.9.2 Object Detection

User states "identify object" or "what is this thing." The obtained image is transmitted to `/api/object/detect`, which provides a list of identified objects along with a label, confidence score, and estimated area (e.g. “left”, “center”, “right”). The application combines the top three outcomes and articulates them in one sentence, for example, “chair on the left, 92 percent. table in the center, 87 percent.”

9.9.3 Currency Detection

User states “identify currency” or “what's the amount of money.” The image is transmitted to `/api/currency/analyze`, which provides the count of banknotes found and their overall worth. The assistant states the total sum and the count of notes, for example. “Found 500 rupees, consisting of 2 notes.

9.9.4 Face Recognition and Enrolment

User asks "who is this" or "identify face." The Faces screen takes a picture with the shared camera and transmits it to `/api/face/analyze`. Names that are recognized are spoken out loud; if a match isn't found, the count of detected (yet unrecognized) faces is provided. The same interface also offers an enrollment process: the user (or a sighted helper) inputs a name and takes a photo, which is transmitted to `/api/face/register` for future recognition of the individual.

9.9.5 Scene Description

The user asks, “explain the surroundings” or “what is nearby.” The picture is transmitted to `/api/blind-assist/analyze`, which provides a summary in natural language created by a vision-language model on the server side. This overview is addressed directly to the user, providing a broad depiction of their environment instead of enumerating specific items.

9.10 Conclusion:

The frontend provides a hands-free, voice-controlled interface for users with visual impairments. An asynchronous voice loop enables wake-word detection and command processing without touch input, while an Android foreground service keeps the camera and microphone active in the background or when the screen is locked. The frontend communicates with FastAPI through dedicated service layers to provide real-time audio responses for text reading, object, currency, face, and scene recognition. Accessibility features, such as semantic labels, high-contrast controls, and live announcements, improve usability and support independent daily living.

Chapter 10 Conclusions

10.1 Summary

This project focused on the design and development of **See Beyond**, an assistive vision-based system intended to support visually impaired individuals in understanding their surrounding environment. The work concentrated on implementing and evaluating core perception modules using computer vision and deep learning techniques in an offline setting.

The system development included the implementation of **object recognition**, **Pakistani currency detection**, **people (face) recognition**, and the **research-driven design of an OCR module**. Object and currency recognition were implemented using YOLO-based deep learning models, with extensive dataset preparation, preprocessing, augmentation, and performance evaluation. The currency detection module achieved high accuracy through the creation of a custom dataset and careful model selection and training.

The people recognition module was implemented using an embedding-based face recognition approach. Facial embeddings were generated using a FaceNet-based model and matched against a local database using cosine similarity, enabling real-time recognition without storing raw facial images. This approach demonstrated scalability, storage efficiency, and privacy preservation during testing.

Overall, the work completed in this project demonstrates the feasibility of using lightweight deep learning models and local processing to perform essential assistive perception tasks. The implemented modules provide a solid technical foundation for developing a fully integrated assistive system in future phases.

10.2 Recommendations for Future Work

While the current project successfully implemented core perception modules, several enhancements can be explored to further improve functionality, usability, and real-world deployment:

1. Embedded Hardware Deployment

All implemented modules can be migrated from PC-based prototypes to an embedded platform such as ESP32 or similar edge devices, with optimization for power consumption and real-time performance.

2. Navigation and Localization Features

Indoor and outdoor navigation capabilities can be added using GPS, BLE beacons, or vision-based SLAM techniques to provide directional guidance and route assistance.

3. Dataset Expansion and Model Refinement

Object and currency datasets can be expanded to include additional classes, environmental conditions, and real-world variations to improve robustness and generalization.

4. User Testing and Usability Evaluation

Extensive testing with visually impaired users can be conducted to evaluate system usability, accuracy, and long-term effectiveness in real-world scenarios.

5. Security and Privacy Enhancements

Additional privacy-preserving mechanisms such as encryption of stored embeddings and access control for system updates can be incorporate

References

- Cs, A., M, D., R, H. V., & Anju, A. (2022). Currency Recognition For The Visually Impaired People. *2022 IEEE Delhi Section Conference (DELCON)*, 1–3. <https://doi.org/10.1109/DELCON54057.2022.9753373>
- Ghodake, A., Sale, H., Kamble, A., Mankari, K., Lad, O., & Mishra, V. (2023). Facial Recognition Smart Glasses for Visually Impaired People. *2023 7th International Conference On Computing, Communication, Control And Automation (ICCUBEA)*, 1–5. <https://doi.org/10.1109/ICCUBEA58933.2023.10391977>
- Joshi, R. C., Yadav, S., & Dutta, M. K. (2020). YOLO-v3 Based Currency Detection and Recognition System for Visually Impaired Persons. *2020 International Conference on Contemporary Computing and Applications (IC3A)*, 280–285. <https://doi.org/10.1109/IC3A48958.2020.233314>
- Kai Wang, Babenko, B., & Belongie, S. (2011). End-to-end scene text recognition. *2011 International Conference on Computer Vision*, 1457–1464. <https://doi.org/10.1109/ICCV.2011.6126402>
- Koch, G., Zemel, R., & Salakhutdinov, R. (n.d.). *Siamese Neural Networks for One-shot Image Recognition*.
- Yang, J., Ren, P., Zhang, D., Chen, D., Wen, F., Li, H., & Hua, G. (2017). Neural Aggregation Network for Video Face Recognition. *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 5216–5225. <https://doi.org/10.1109/CVPR.2017.554>
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection* (No. arXiv:1506.02640). arXiv. <https://doi.org/10.48550/arXiv.1506.02640>

Zhao, Z.-Q., Zheng, P., Xu, S.-T., & Wu, X. (2019). Object Detection With Deep Learning: A Review. *IEEE Transactions on Neural Networks and Learning Systems*, 30(11), 3212–3232. <https://doi.org/10.1109/TNNLS.2018.2876865>

Tapu, R., Mocanu, B., & Zaharia, T. (2020). Wearable assistive devices for visually impaired: A state of the art survey. *Pattern Recognition Letters*, 137, 37–52. <https://doi.org/10.1016/j.patrec.2018.10.031>

Zhou, X., Yao, C., Wen, H., Wang, Y., Zhou, S., He, W., & Liang, J. (2017). EAST: An Efficient and Accurate Scene Text Detector. *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2642–2651. <https://doi.org/10.1109/CVPR.2017.283>

Ahmad, A., Dubey, R., & Kumar, S. (2018). Smart cane for visually impaired people using ultrasonic and IR sensors. *International Journal of Engineering Research & Technology*, 7(4), 1–5.

Ali, M., Hur, S., & Park, Y. (2019). Wi-Fi fingerprinting localization for indoor navigation. *IEEE Access*, 7, 111215–111225. <https://doi.org/10.1109/ACCESS.2019.2934511>

Borenstein, J., & Ulrich, I. (1997). The GuideCane—A computerized travel aid for the active guidance of blind pedestrians. *IEEE Robotics & Automation Magazine*, 4(1), 45–52. <https://doi.org/10.1109/100.580977>

Campaña, A., Bergasa, L. M., Alcantarilla, P. F., & Yebes, J. J. (2017). BLE-based indoor positioning for assistive navigation. *Sensors*, 17(12), 1–20. <https://doi.org/10.3390/s17122890>

Ghodake, A., Patil, S., Jadhav, P., & Kulkarni, P. (2023). Facial recognition smart glasses for visually impaired people. In *Proceedings of the 7th International Conference on Computing, Communication, Control and Automation (ICCUBEA)* (pp. 1–6). IEEE. <https://doi.org/10.1109/ICCUBEA58933.2023.10391977>

Gozick, B., Subbu, K. P., Dantu, R., & Maeshiro, T. (2011). Magnetic maps for indoor navigation. *IEEE Transactions on Instrumentation and Measurement*, 60(12), 3883–3891. <https://doi.org/10.1109/TIM.2011.2158442>

Koch, G., Zemel, R., & Salakhutdinov, R. (2015). Siamese neural networks for one-shot image recognition. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*.

Meshram, A., Patil, R., & Bhaskar, P. (2019). NavCane: A smart electronic travel aid for visually impaired. *International Journal of Advanced Research in Computer Science*, 10(3), 45–51.

Microsoft. (2021). *Seeing AI*. <https://www.microsoft.com/seeing-ai>

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 779–788). <https://doi.org/10.1109/CVPR.2016.91>

Sivaprasad, S., Kumar, V., & Reddy, S. (2024). Smart goggles for the visually impaired. In *Proceedings of the IEEE International Conference on Power, Energy, Control and Transmission Systems (ICPECTS)*. IEEE.

World Health Organization. (2023). *World report on vision*. WHO Press.

Yang, Y., Liu, Y., & Song, M. (2017). Neural aggregation network for video face recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4362–4371). <https://doi.org/10.1109/CVPR.2017.464>

Rutuja Bankar, Nikita Bargat, Isha Hanmante, & Prof. Hemlata Dakore. (2022). Face Recognition Using Facenet Deep Learning Network for Attendance System. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, 458–463. <https://doi.org/10.32628/CSEIT228630>

Kim, J. (2025). Multi-sensory Feedback-assisted Low-cost Smart Walking Cane Forming a Wearable Assembly for Blind People. *JOURNAL OF SENSOR SCIENCE AND TECHNOLOGY*, 34(1), 27–36. <https://doi.org/10.46670/JSST.2025.34.1.27>

Aung, S. T., Funabiki, N., Aung, L. H., Kinari, S. A., Mentari, M., & Wai, K. H. (2024). A Study of Learning Environment for Initiating Flutter App Development Using Docker. *Information*, 15(4), 191. <https://doi.org/10.3390/info15040191>